Introduction **5**
Foreword 6
Abstract 7
German 7
Italian 7
English 8
Motivation 9
Name 9
Why Rust? 9
Declaration of Independence 10
License 11
Hardware **13**
Block diagram 15
Processor 16
Schematic 17
Processor 17
Power supply 19
SD card 22
Boot Pins 24
Circuit board 26
Software **35**
GitHub Repository 36
The Arm Cortex A8 36
The CPU 38
CPU Modes 38
Registers 40
Coprocessor 15 42
Vectors 42
The MMU 43
Page Tables 43
Level 1 44
Level 2 45
Translation Lookaside Buffer 46
Boot 47
Compilation 47
Linker script 48
Cargo 49
Build Script 49
Startup 51
Stack Setup 52
Cache Setup 53
Exceptions Setup 54
AM335 Hardware 55
Interrupts 56

Enabling 56
Handler 57
Clocks 59
Pin multiplexing 60
GPIO 61
Initialization 61
Pin Mode 61
Pin Writing 62
Pin Read 62
Interrupts 62
Initialization 62
Registration 63
Handling 64
I2C 65
Initialization 66
Operation 67
Transmit 67
Interrupts 67
XRDY 69
XDR 69
ARDY 70
NACK 71
Timer 72
Initialization 73
Interrupts 74
Modularity 74
Kernel 76
Sysclock 77
Syscalls 78
Handler 79
Rust Handler 81
Signature 81
Logic 82
Syscall Enum 83
Multitasking 85
Preemptive Multitasking 86
Cooperative Multitasking 89
Task Recovery 91
Scheduling 92
Paging 96
Initialization 97
L1 Page Table 98
L2 Page Table 99

Creating an L2 Page Table 100
Application 102
Adding Tasks 103
User Binary 106
Structuring 107
Heap 108
Kernel 109
Libfenix 110
Project Management **113**
Project Proposal 115
Requirements Specification 117
Gantt chart 119
Cost calculation **123**
Consulting Hours 124
Working hours 124
Circuit board 125
BOM 126
Total Expenses 127
List of Figures **129**
Conclusion **133**

Introduction

Foreword

Dear Reader,

My name is Felix Salcher, and I am currently a 11th-grade student at the “Oskar von Miller” Technical High School in Merano, specializing in electronics and electrical engineering.

This documentation describes my senior project, in which I independently developed my own operating system. The project involved implementing both hardware and software components and subsequently presenting the results. We had five class hours per week available for the implementation. However, since my project was very extensive, I invested the majority of the work in my free time as well.

During the development process, I gained valuable experience, particularly regarding how operating systems function and the architecture of the CPU. Additionally, the project sparked my enthusiasm for hardware-oriented programming—an area in which I previously had little experience, as I had mainly programmed in the server sector. I am convinced that this project has provided me with valuable experience for my future career path.

I hope this documentation provides a small glimpse into the world of operating systems and perhaps also sparks your interest in this fascinating topic.

Abstract

German

The goal of my high school graduation project was to develop a minimalist kernel for a microcontroller. No external code was allowed to be used, so the entire logic had to be implemented from scratch. In addition, I designed a custom circuit board equipped with an ARM Cortex processor.

The kernel performs key operating system functions such as multitasking, which allows multiple processes to run in parallel. Each task runs in user mode, is assigned a fixed memory area in RAM, and is thus prevented from accessing other memory addresses—particularly those of the kernel. This method of memory isolation is known as paging.

Communication between the kernel and tasks takes place via system calls. Furthermore, the kernel provides basic interfaces for controlling peripherals, such as GPIO pins or the I2C bus.

Italian

The goal of my senior project was to develop a minimalist kernel for a microcontroller. No external libraries were used to achieve this: all the code was written manually. In addition, a dedicated electronic board with an ARM Cortex processor was designed.

The kernel implements fundamental operating system functions, including multitasking, which allows for the concurrent execution of multiple tasks. Each task runs in user mode and has its own region of RAM, without the ability to access unauthorized memory spaces, thus ensuring kernel protection. This technique is known as paging.

Communication between the kernel and the tasks occurs via system calls (syscalls). Additionally, the kernel provides interfaces for managing key peripherals, such as GPIO and the IZC bus.

English

The goal of my final project was to develop a minimalist kernel for a microcontroller. No external libraries were used; instead, all code was written from scratch. To support the implementation, a custom board with an ARM Cortex processor was designed.

The kernel implements essential operating system functions, including multitasking, allowing multiple tasks to run concurrently. Each task is executed in User Mode and is assigned a dedicated block of memory, preventing unauthorized access to other memory regions and ensuring kernel protection. This mechanism is known as paging.

Communication between the kernel and user-mode tasks is implemented through system calls (syscalls). In addition, the kernel provides interfaces for controlling peripherals such as GPIO and the IZC bus, enabling interaction with external hardware.

Motivation

I decided to develop my own operating system as my high school graduation project because this subject area has fascinated me for quite some time. By implementing it on my own, I wanted not only to expand my knowledge in the field of software development but also to gain a deeper understanding of the structure and functioning of processors. Such a project proved particularly suitable for this purpose, as it combines both theoretical and practical aspects of computer science.

Name

When choosing a name, I took inspiration from Linux, which was named after its developer, Linus Torvalds. Therefore, I decided to name my project Fenix. Originally, it was titled MicroOS, but this seemed too generic to me, which is why I ultimately decided to rename the project. Consequently, the old name still appears in some documents.

I chose a phoenix as the logo because its English pronunciation matches the name of my project.

Why Rust ?

Since we only use C++ and assembly language at school, Rust might seem like an unusual choice of programming language for this project. I chose Rust because I always enjoy trying out new languages. For example, I used Go as the programming language for my 2023/24 Füla project so I could learn it. Rust has several features that really set it apart from C-based languages like C++ or C.
Here are what I consider to be the most important features of Rust for this project:

- Memory **Safety:** Probably the most important feature of Rust is its guaranteed memory safety. Rust uses the “Borrow Checker,” which checks the written code to ensure that static rules are followed, allowing the compiler to guarantee memory safety. Problems eliminated by this include: null pointers, use-after-free, dangling pointers, and race conditions.
- **Error Handling:** In many C-based languages like C++, try-catch is used to handle errors. However, this is not ideal because error checking is optional. In Rust, on the other hand, every error must be checked before the value can be used.
- **Type System:** Rust’s type system is another reason to use Rust. Enums in Rust, in particular, are a major advantage and are frequently used. The *`match`* statement in Rust is particularly useful; unlike the *`switch`* statement, it can return a value directly. Above all, “exhaustive matching”—where all possibilities must be checked—is a significant advantage.
- **Tooling:** Unlike the C ecosystem, which has a good dozen different tools for compilation, LSP, formatters, compilers, etc., Rust has all the necessary tools already integrated, which makes software development significantly easier.

Declaration of Independence

I, Felix Salcher, a student at the “Oskar von Miller” Technical High School in Merano, specializing in electronics and electrical engineering, hereby certify that I have completed this senior thesis, titled “Fenix,” independently and without unauthorized assistance from third parties. All ideas, data, and text passages that I have taken verbatim or in essence from books, journals, internet sources, or personal conversations are marked as such in the text and listed in full in the bibliography and list of sources. I
further affirm that, apart from the persons and resources mentioned in the preface, I have not sought any further assistance and that I have not submitted this thesis, in this or any similar form, to any other examining authority.

Merano, May 19, 2025

Project Supervisor, Felix Salcher

_____

Subject Teacher: De Tomaso Martin

_____

Subject Teacher: Ivan Huber

_____

Principal: David Augscheller

_____

License

This project (including source code and documentation) is licensed under the GNU General Public License (GPL) Version 3.0. This license grants all users extensive rights, such as the freedom to:

- Use the project for any purpose.
- To examine how it works and adapt it to your own needs.
- Make changes and develop improvements.
- Distribute copies of the original or the modifications.

However, as a central condition (copyleft), the GPL requires that any distribution of the project or works derived from it must be done under the same license terms. This ensures its permanent status as free software.

The full license can be accessed in the GitHub repository at the following link: https://github.com/salfel/fenix/blob/master/LICENSE

Hardware

At the start of the project, I had intended to use TI's AM62 processor. However, after several weeks of developing this processor, I decided to use the far simpler AM335. This was primarily because, when developing the AM62, I had to design the entire board from scratch, meaning I had to research and design everything myself. Since I had no experience in
Given the challenges involved in developing such complex projects, I eventually had to abandon the AM62 after a few weeks because its development was taking too much time.
Instead, I decided to use the AM335, since the *Beaglebone Black*, which is based on this processor, had already been developed. My board was built on its design, with only the choice of components matching that of the *Beaglebone*; I routed the board myself.
It should be noted that the board was designed very minimally to save development time and, above all, to create a prototype on which the operating system could run.

Block Diagram

The following figure shows the block diagram of the board.

The PMIC is powered via USB. It then provides the necessary voltages. Among other things, it can also be programmed via I2C and supplies the micro SD card with 3.3 V.
The microSD card connects a total of four data lines to the processor. These include the clock signal for the microSD card and the CMD line, through which the processor sends commands to the microSD card.
The boot pins and the pin header are also shown here. The boot pins are responsible for telling the processor which device to use for booting, since it has various

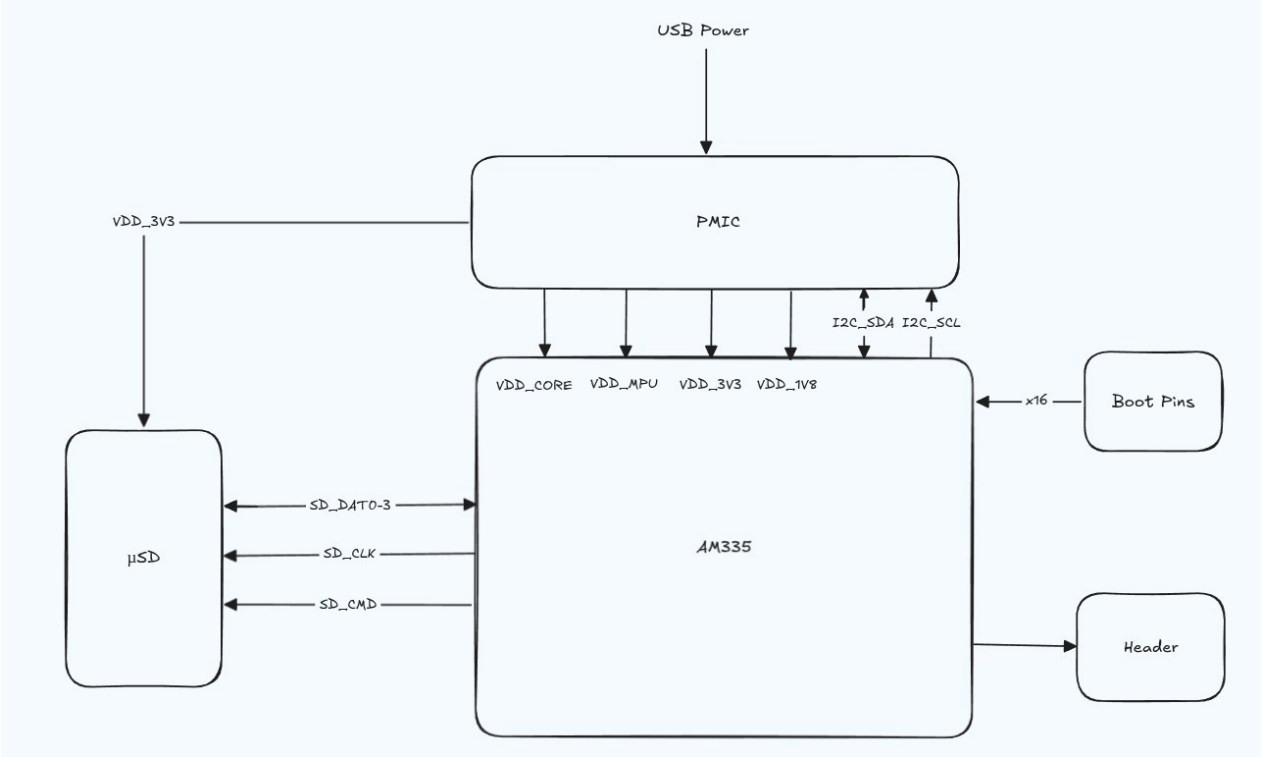


Figure 1. Block diagram of the board

Processor

A processor from the AM335 family by Texas Instruments was used as the project's processor. It is based on the ARM Cortex-A8 architecture and has a clock frequency of up to 1 GHz.

The AM335 offers a wide range of applications and can be used in industrial automation, medical technology, and control interfaces, among other fields. The AM335's major advantage lies in its excellent balance of computing power, cost-effectiveness, energy efficiency, and its extensive range of peripherals—including UART, I2C, USB, CAN, and many more—making it particularly attractive for industrial applications.

Among other things, the AM335 isn't limited to industrial applications. It has enough power to run operating systems like Linux. Thanks to boards like the Beaglebone Black, which the AM335 , can it among among other thingsalso be used as a desktop processor. By implementing HDMI or LCD, a display can be controlled; some processors in the AM335 family also feature a GPU to optimize the display of GUIs.

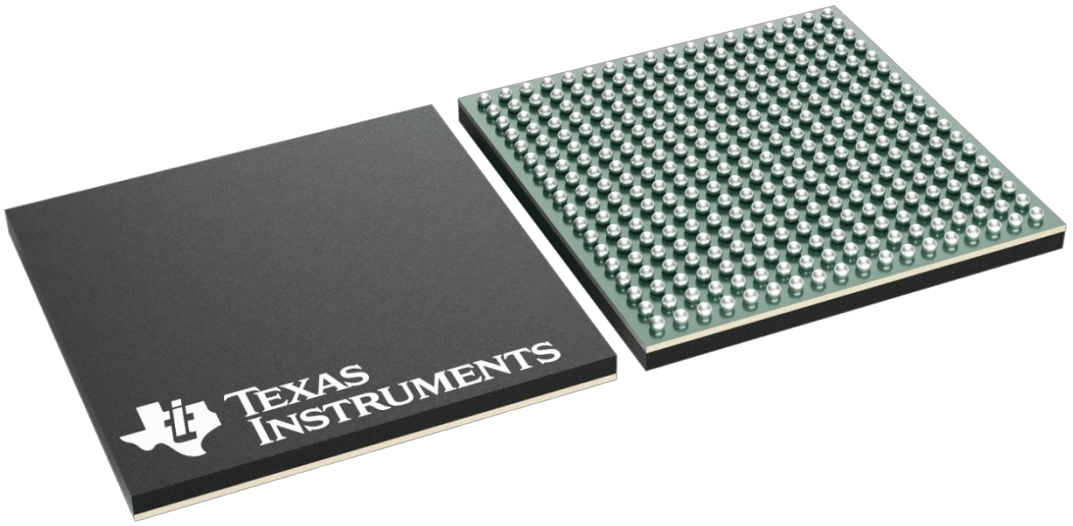


Figure 2. AM335 processor https://www.ti.com/product/AM3358

Schematic

Processor

The project used a processor from the AM335 family by Texas Instruments. It is based on an ARM Cortex-A8 and has a clock frequency of up to 1 GHz. It includes a number of interfaces and peripherals, including USB, UART, I2C, Ethernet, CAN, and GPIO, among others.

The following figure shows the processor's schematic sheet. It includes:

- The crystal, which provides the processor with a constant clock frequency of 25 MHz, is not the processor's primary clock frequency.
- A reset button to reset the processor.
- A pin header for debugging various signals.
- Wires that connect the processor to various peripherals, such as I2C and GPIO.

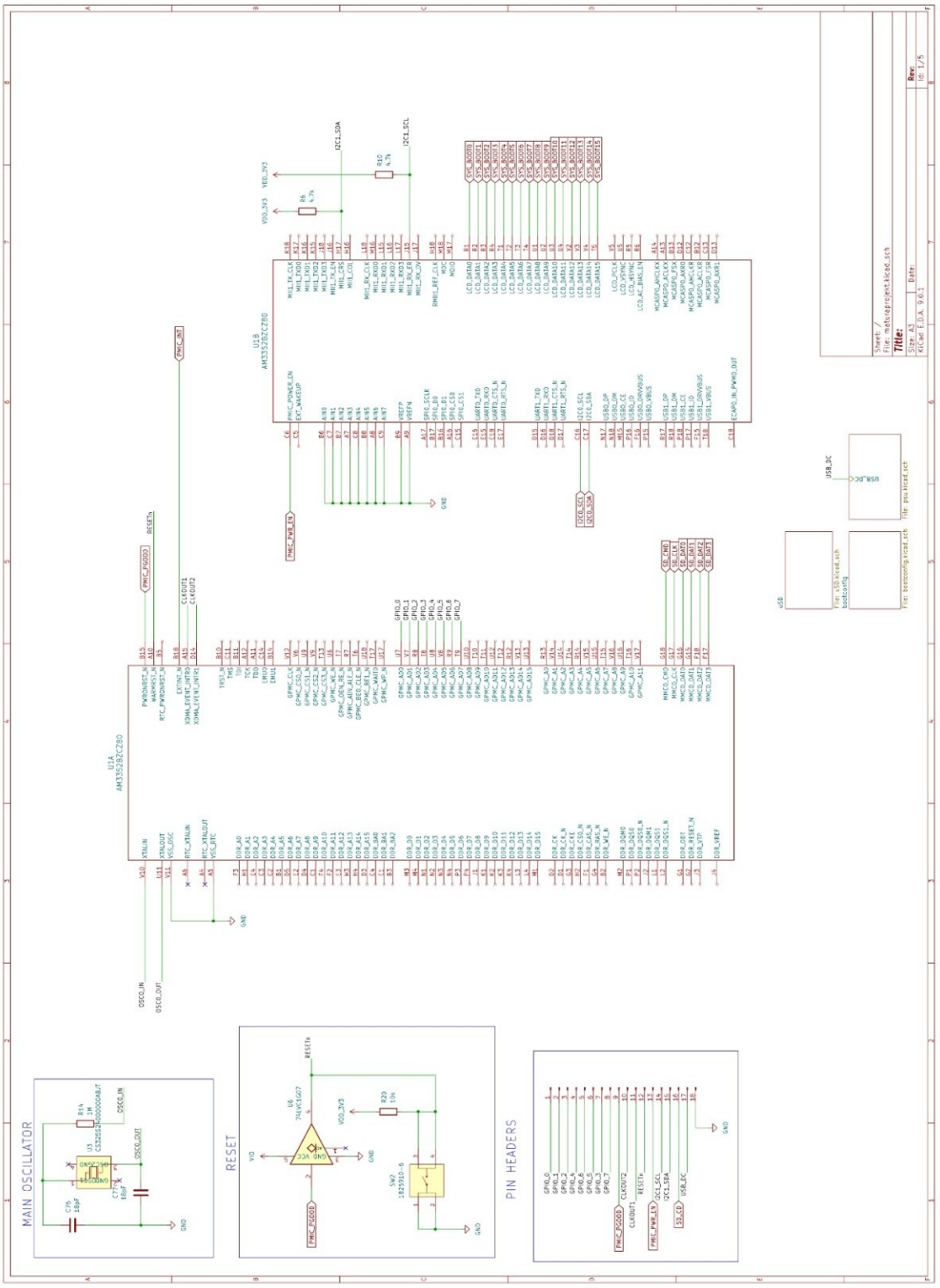


Figure 3. Processor Schematic

Power Supply

The *TPS65217* was used as the PMIC. This is the recommended PMIC for the AM335, but another one can also be used. However, the *TPS65217* is by far the best option, as its compatibility with the AM335 has already been extensively tested. Among other features, it includes 3 step-down converters and 4 LDOs. Additionally, a battery can be connected to the PMIC, which then manages its charging and discharging. It also provides mechanisms for supplying power via an external power supply. Furthermore, certain functions of *the TPS65217* can be programmed via I2C. To monitor voltage stability, the PMIC also provides 2 signals that allow the processor or external components to perform specific actions if the supply voltage becomes unstable.

The required signals are then connected to the processor, whereby a number of support capacitors are needed for the various voltages to keep the voltage as stable as possible during dips.

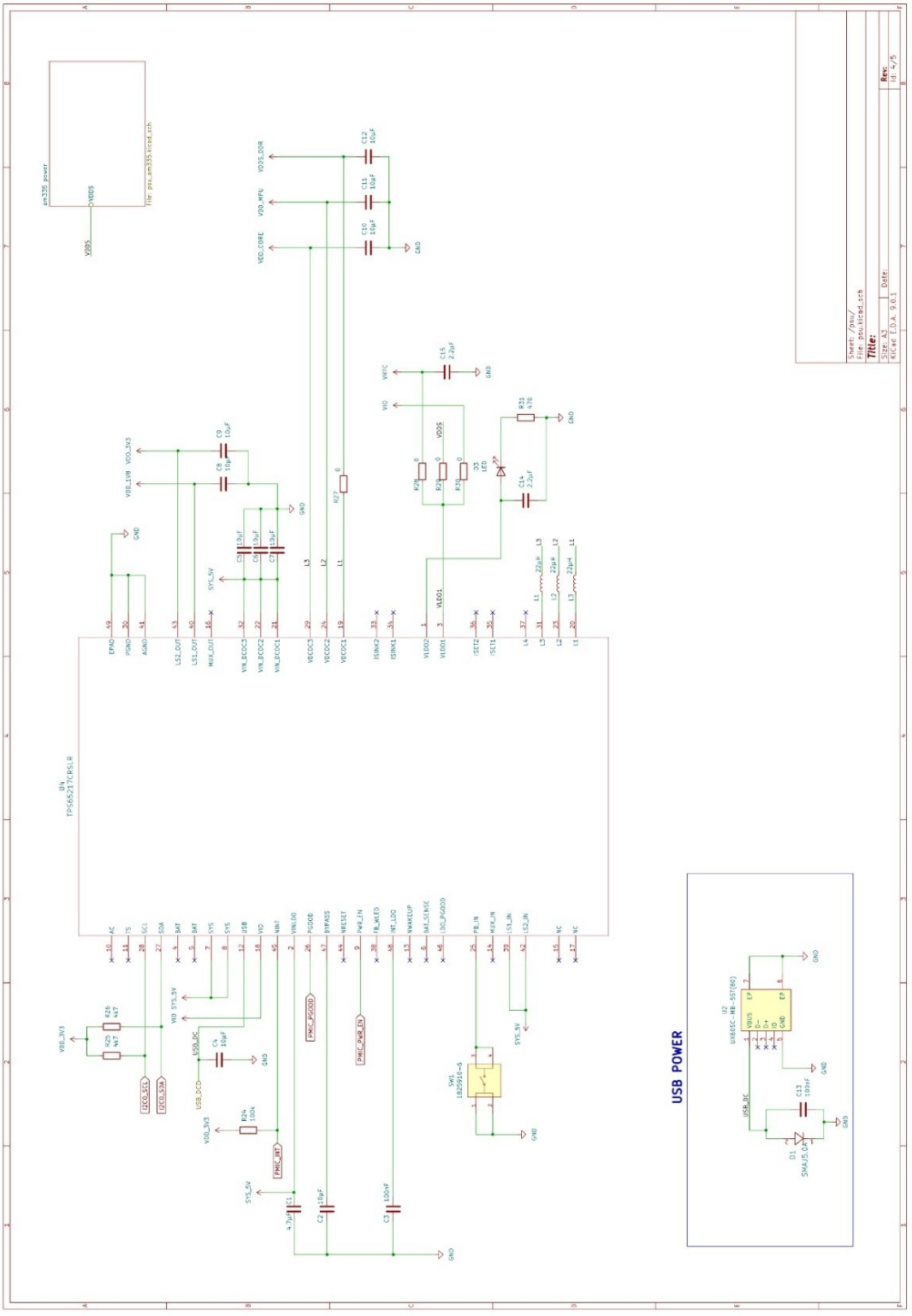


Figure 4. PSU Schematic

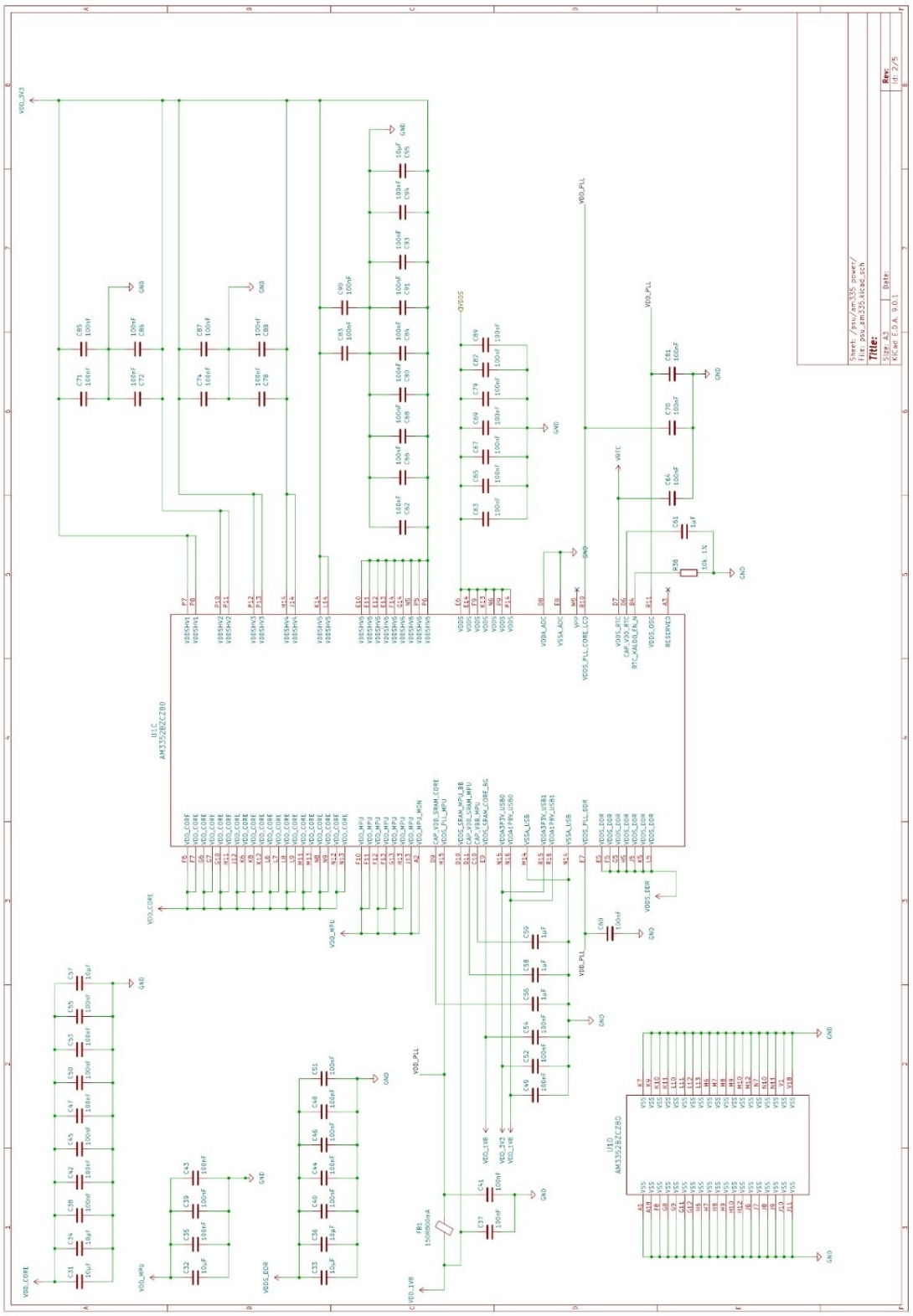


Figure 5. PSU AM335 Schematic

SD- card

The AM335 offers a variety of options for booting the processor, including USB, UART, Ethernet, flash memory, and a micro SD card. In this project, the micro SD card was used to boot the kernel, as this is the simplest option to implement. Since the processor cannot know which interface to use for booting, this must be communicated to it via a configuration of resistors acting as inputs.

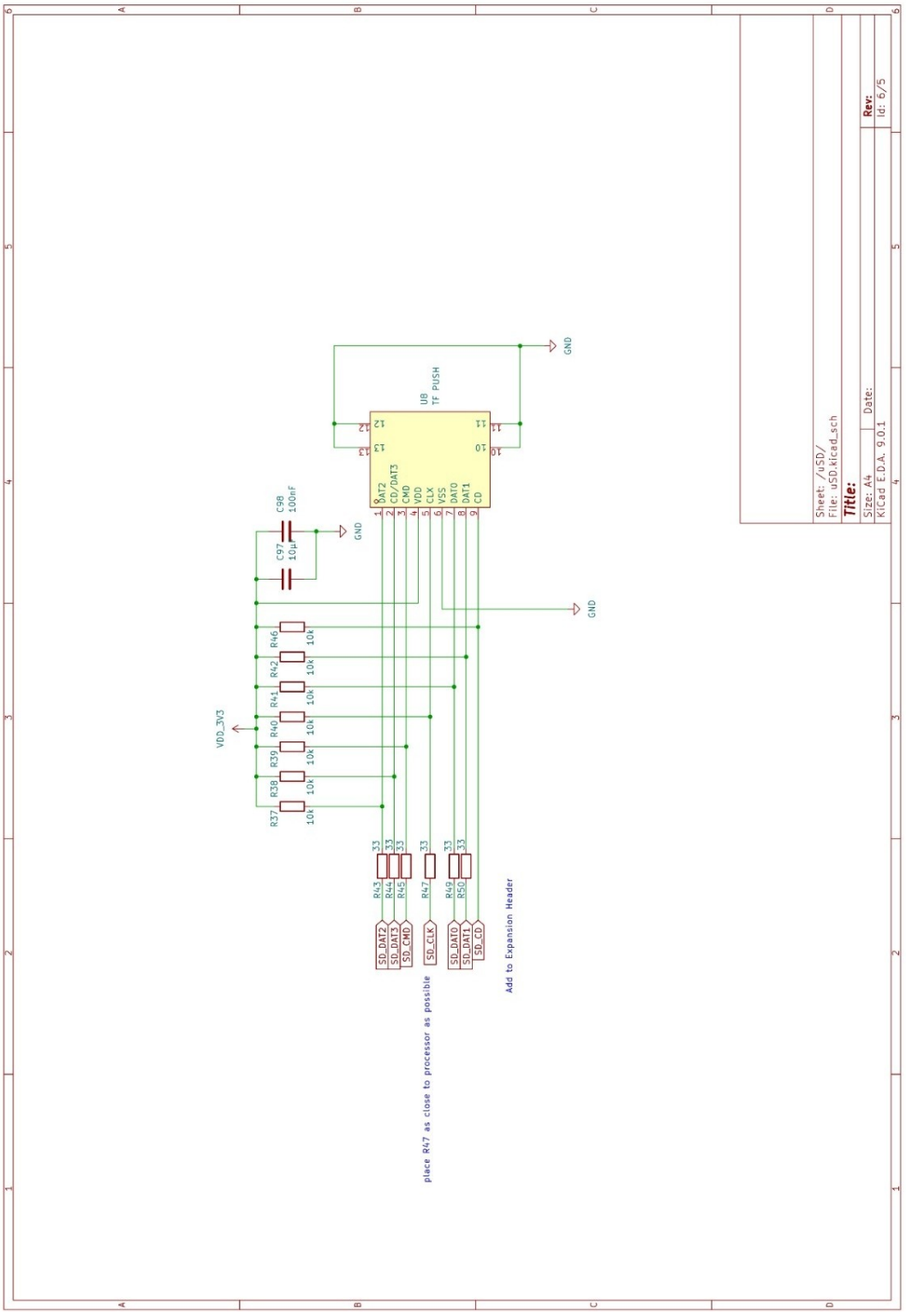


Figure 6. SD Card Schematic

Bootpins

In order for the processor to use the SD card for booting, the boot pins must be set correctly. In addition to the boot sequence, you can also select the desired crystal frequency, among other settings. The other bits are not relevant for the following boot sequence and are therefore set to *0*.
A total of 31 different boot sequences can be selected. If the first peripheral fails to boot the processor, the system attempts to boot via the next peripheral. One advantage of this method is that, for example, the processor can be booted via different peripherals with the press of a button, as is the case with the Beaglebone Black, where the processor can be booted via the SD card with a single button press; otherwise, it boots from the flash.

SYSBOOT[15:14]	SYSBOOT[13:12]	SYSBOOT[11:10]	SYSBOOT[9]	SYSBOOT[8]	SYSBOOT[7:6]	SYSBOOT[5]	SYSBOOT[4:0]	Boot Sequence			
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	11100b	MMC1	MMC0	UART0	USB0

Figure 7. Boot configuration used TRM page 5035

The boot pins are configured using a series of resistors, which are connected either to ground or 3.3V, depending on the pin.

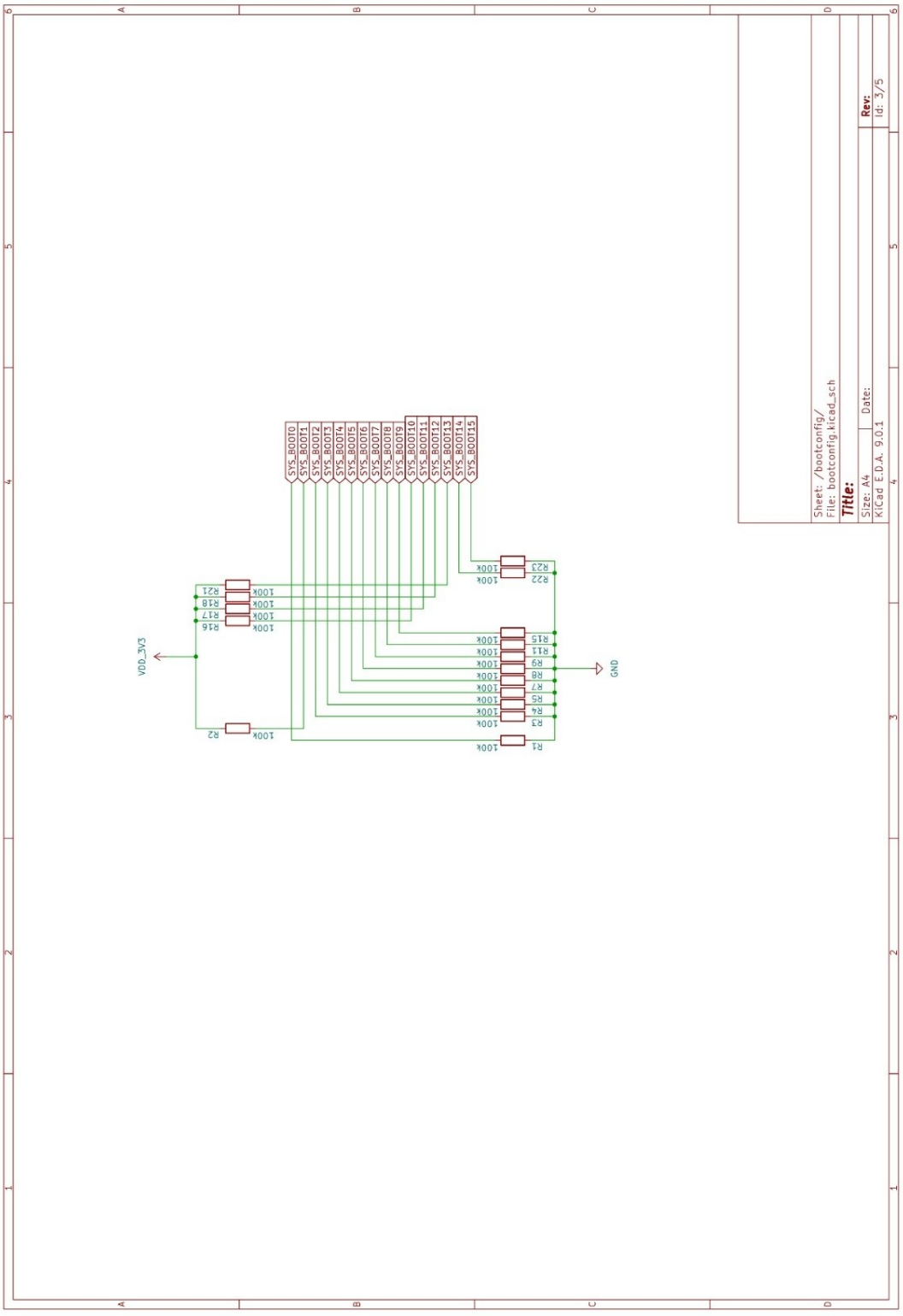


Figure 8. Boot Pins Schematic

PCB

Since the pins for the AM335's power supply are all located in a very small space and also require the largest possible traces or polygons due to heat generation, a total of 6 layers are used on the PCB here. The second layer is used for ground and the fourth layer for the power signals.

The circuit board is shown on the following pages. The first image combines all layers, though ground and power are excluded so as not to obscure the other layers. The subsequent figures show the various layers individually, from Layer 1 to Layer 6.

As you can see, a large number of polygons are required for the processor to connect the various power supplies. On the top layer, where
the processor is mounted, the various pins are connected to the other layers. The necessary support capacitors are also placed on the bottom layer. These have a 0402 package, since a large number of them are required in a very small space.

I then ordered the circuit board from the manufacturer JLCPBC. Among other things, I decided to have the board populated on both sides. This is because the processor's pins cannot be soldered by hand, and the support capacitors are small
and placed in a tight space, which makes soldering very difficult. However, this decision caused the price of the board to skyrocket, mainly because both sides had to be populated, which required a different manufacturing process.

Unfortunately, the board could no longer be tested because JLC soldered together some of the PMIC's pins during assembly. Since this error wasn't noticed at first, the PMIC was short-circuited and no longer worked. Since the board had cost a total of €140, I decided not to reorder it to avoid spending that much money again.

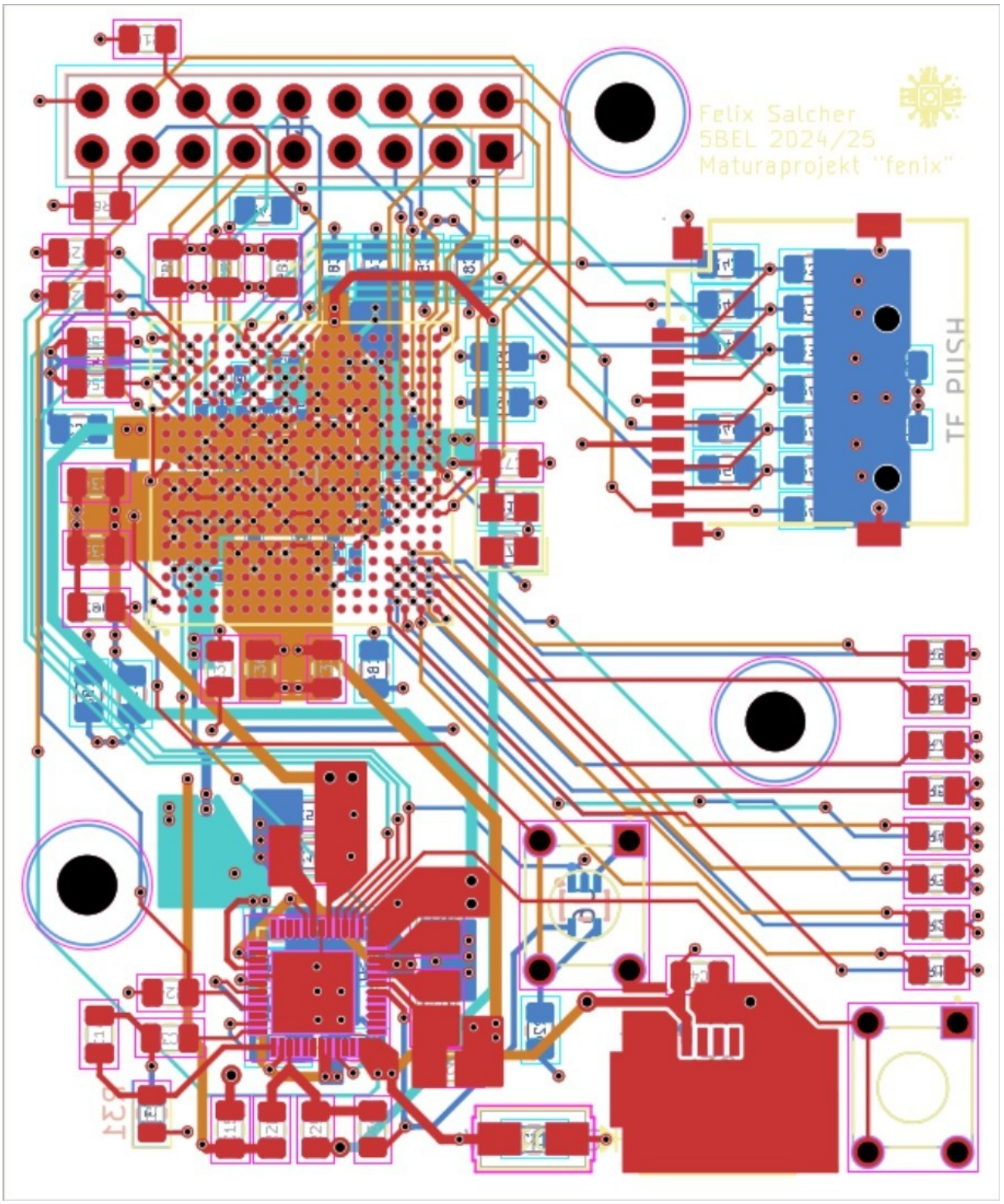


Figure 9. Complete circuit board

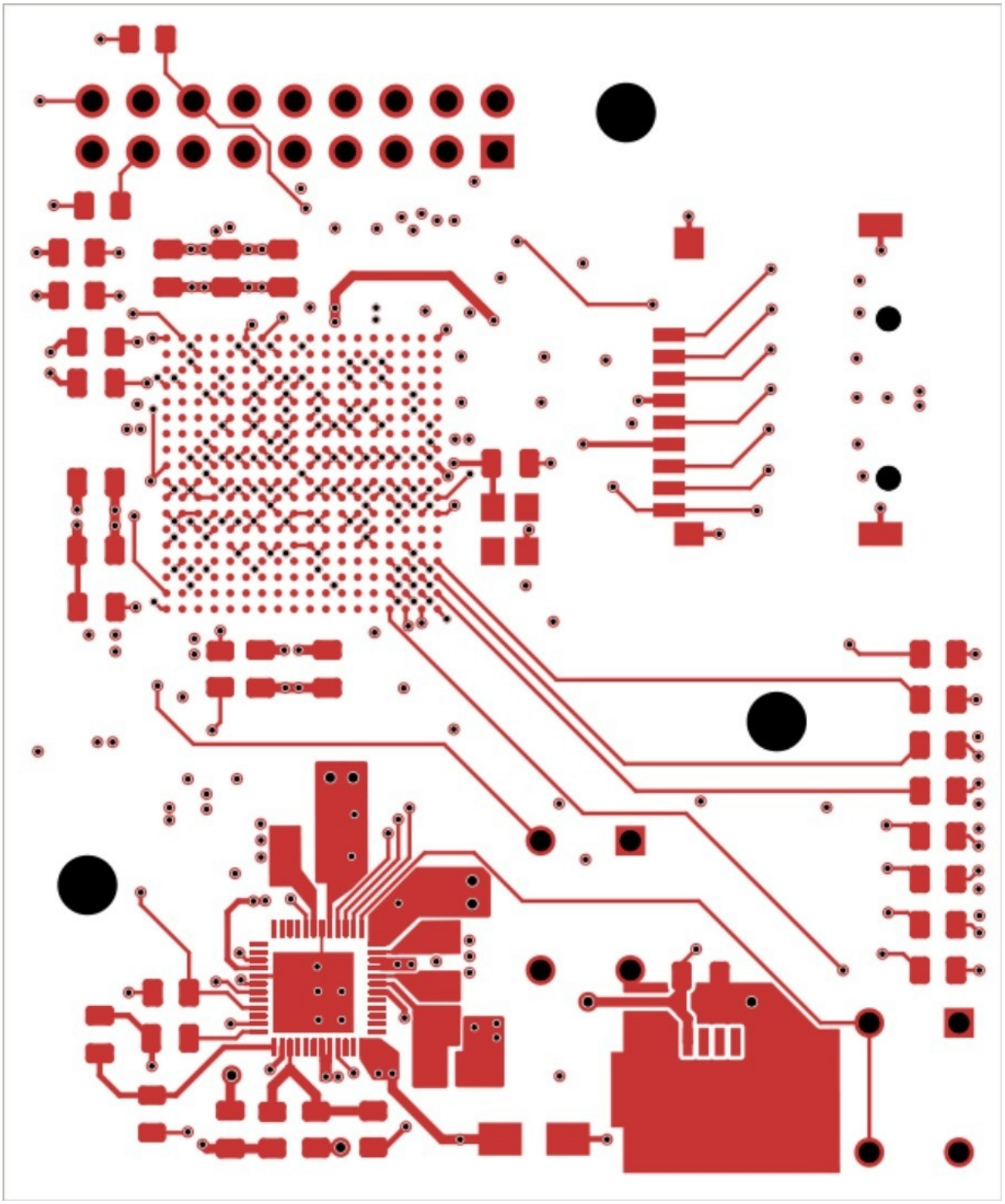


Figure 10. PCB Top Layer

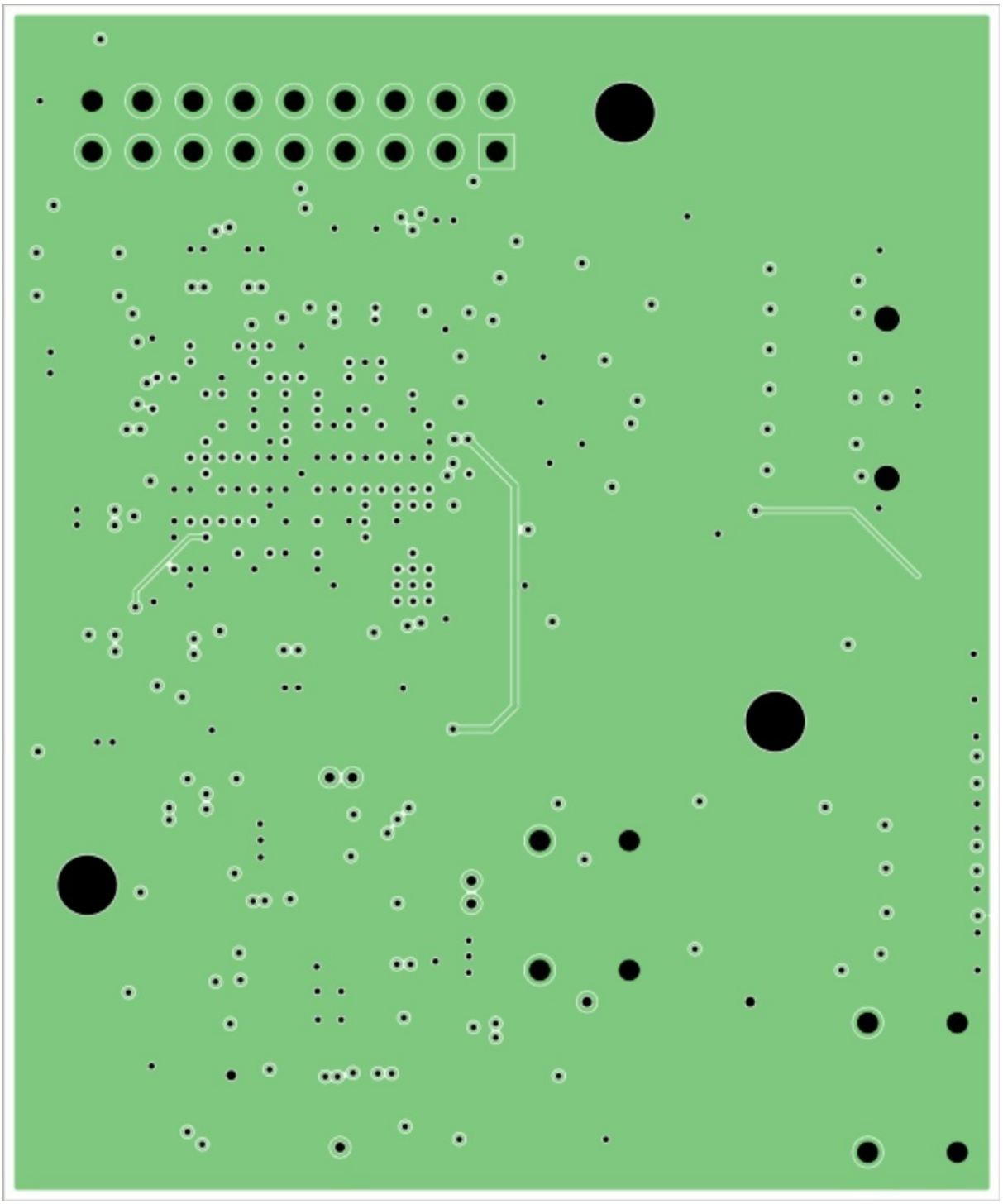


Figure 11. PCB GND Layer

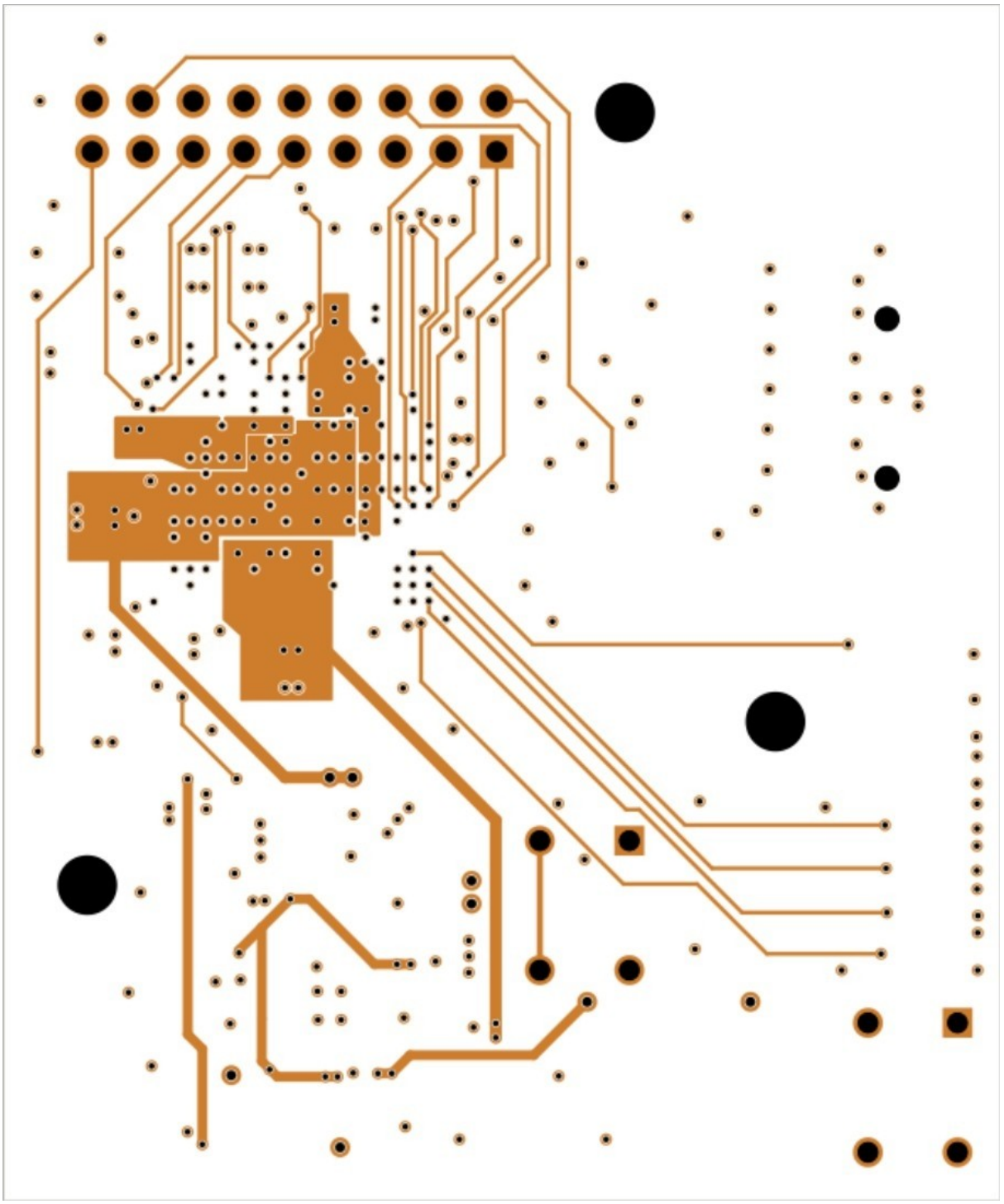


Figure 12. PCB Layer 3

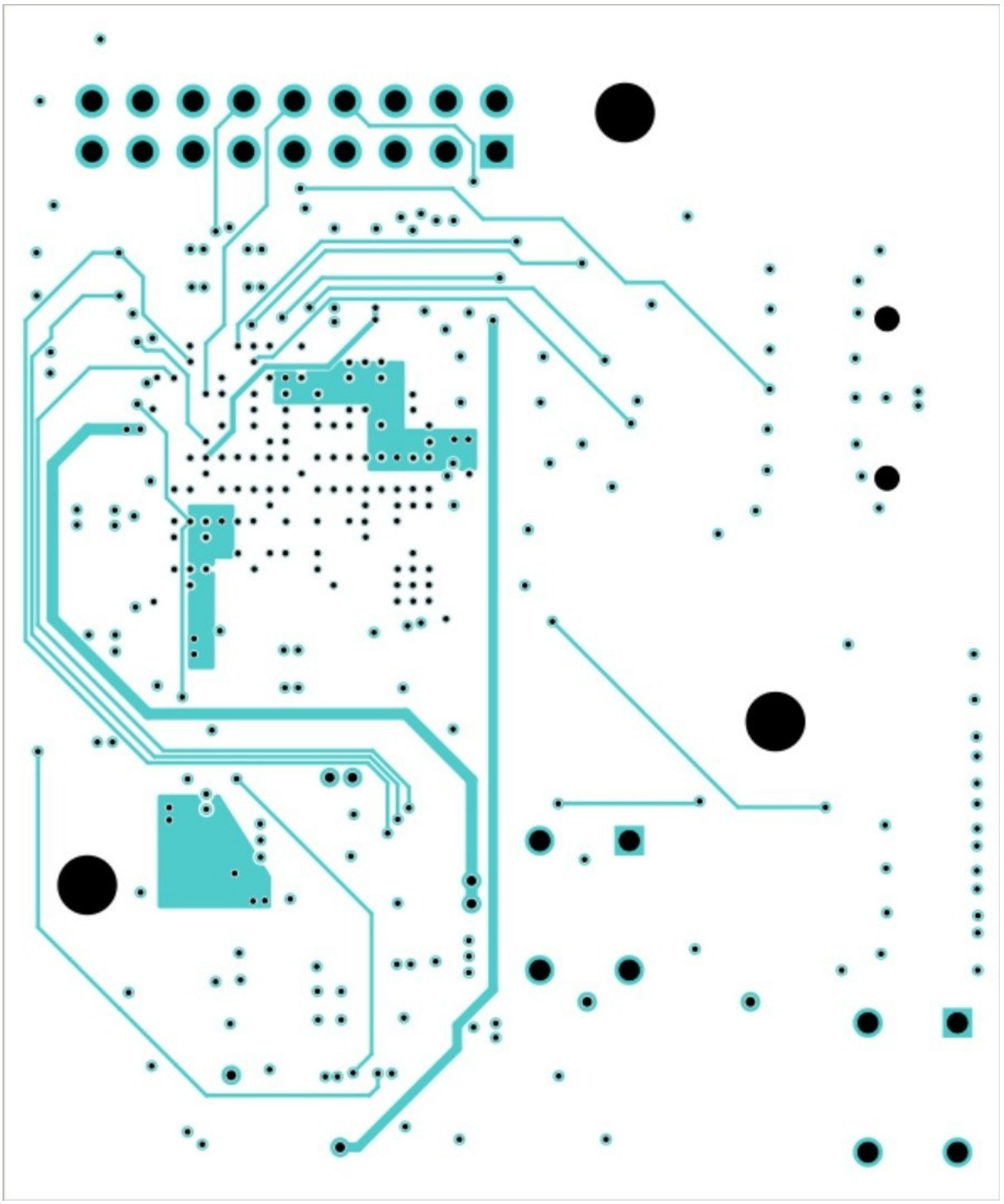


Figure 13. PCB Layer 4

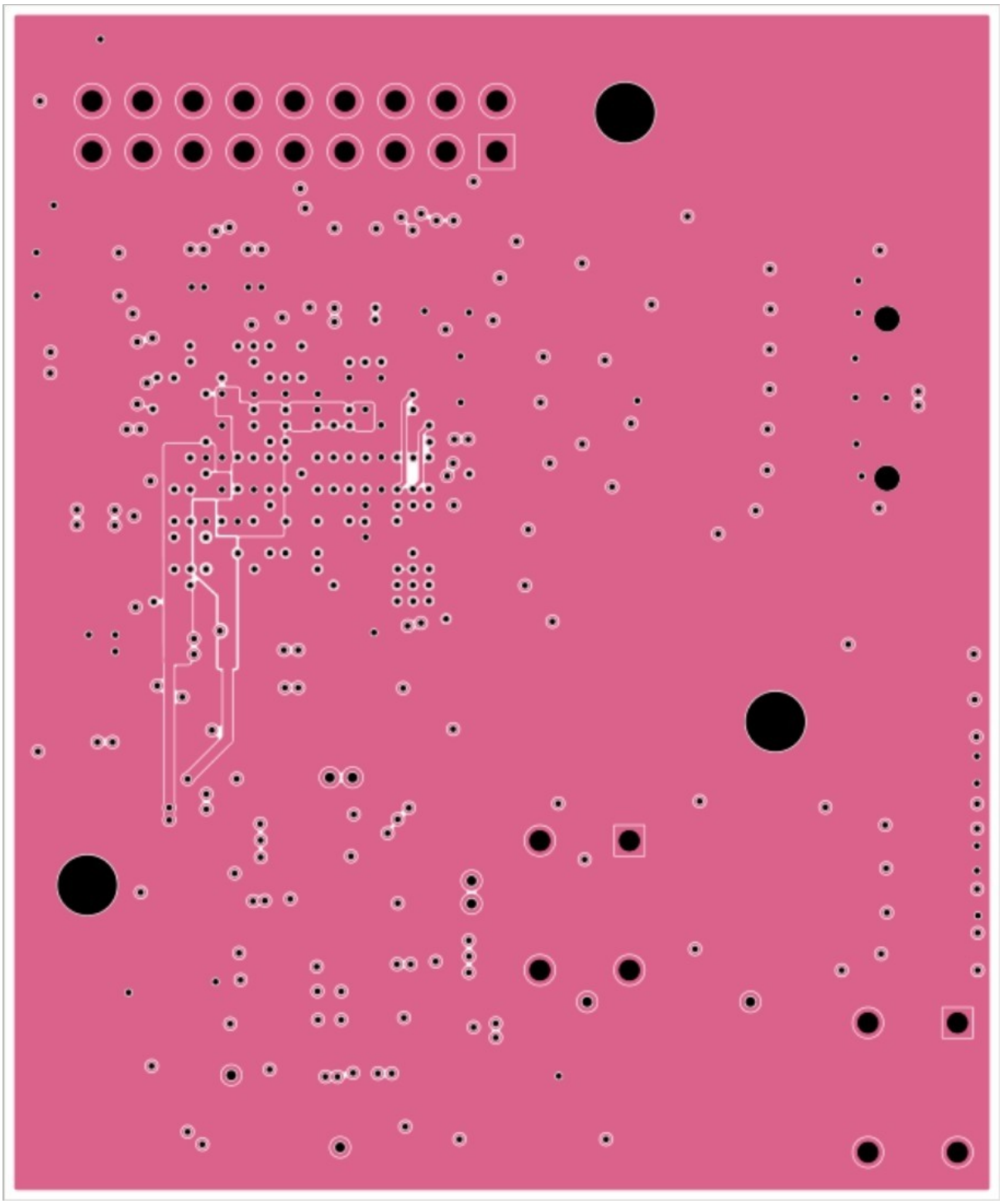


Figure 14. PCB POWER Layer

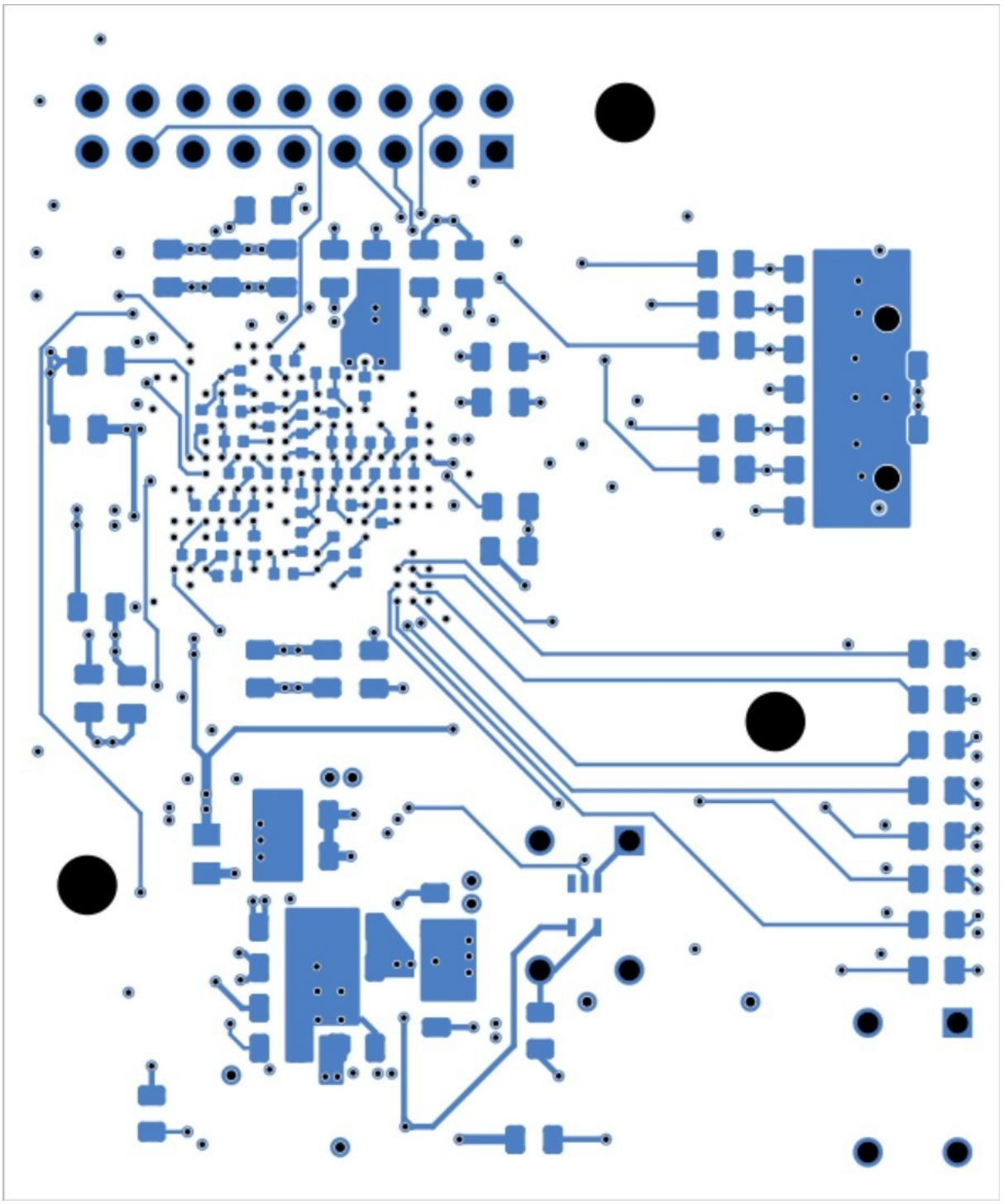


Figure 15. PCB Bottom Layer

Software

GitHub Repository

All of the project's software is stored in an online repository. It can be accessed via the following link: https://github.com/salfel/fenix



Figure 16. Github Repository QR Code

The ARM Cortex-A8

In the following section, I would like to introduce the reader to the ARM Cortex controller, as this is necessary for the subsequent chapters on software in order to better understand them. The controller used here is the TI AM335, which is based on a 32-bit ARM Cortex-A8. In addition to the standard features of the ARM Cortex-A8, it also implements several peripherals such as GPIO, I2C, and many others.
The Arm Cortex A8 is part of the Armv7a architecture. It implements version 7 of the Arm architecture and belongs to the "Applications" family. This is the fastest variant of the Armv7 families, alongside the "Microcontroller" (Armv7-M) and "Real-Time" (Armv7-R) controllers. Features implemented by the Arm Cortex A8 include:

- **L1 and L2 Caches:**
  The Arm Cortex has 32 KB each of L1 data and instruction caches. The data cache is responsible for caching the various address translations performed by the MMU so that so that it does not have to translate the addresses every time they are accessed. The instruction cache is responsible for caching the various instructions so that they do not have to be decoded anew for every instruction. The L2 cache is responsible for providing an additional instruction and data

in case the L1 cache is missed, providing an additional cache where the data may be available.

- **Different CPU Modes:**
  The processor contains a total of 9 CPU modes, each of which performs different tasks. More on this, however, in a later chapter.
- **Memory Management Unit (MMU):**
  The MMU, which is part of every desktop/laptop processor, serves to allocate specific regions of RAM exclusively to certain processes. More on this in a later section

The CPU

CPU Modes

The Arm Cortex-A8 has a total of 9 CPU modes, each of which performs different tasks. Most of these are triggered by so-called exceptions. The figure below shows all of the processor's CPU modes.

Mode	Encoding	Function	Security state	Privilege level
User (USR)	10000	Unprivileged mode in which most applications run	Both	PL0
FIQ	10001	Entered on an FIQ interrupt exception	Both	PL1
IRQ	10010	Entered on an IRQ interrupt exception	Both	PL1
Supervisor (SVC)	10011	Entered on reset or when a Supervisor Call instruction (SVC) is executed	Both	PL1
Monitor (MON)	10110	Implemented with Security Extensions. See Chapter 21	Secure only	PL1
Abort (ABT)	10111	Entered on a memory access exception	Both	PL1
Hyp (HYP)	11010	Implemented with Virtualization Extensions. See Chapter 22	Non-secure	PL2
Undef (UND)	11011	Entered when an undefined instruction executed	Both	PL1
System (SYS)	11111	Privileged mode, sharing the register view with User mode	Both	PL1

Figure 17. ARMv7a CPU Modes
https://developer.arm.com/documentation/den0013/d/ARM-Processor-Modes-and-Registers

As can be seen, the monitor and hypervisor modes are only implemented with certain processor extensions and are not present in the AM335.

- **User**: In most operating systems, user mode is responsible for executing so-called user code. This typically includes the applications that a user downloads onto their operating system. User mode is the only CPU mode that is not privileged. This means it cannot switch processor modes; this must be done by
  an SVC. Additionally, the processor cannot access any arbitrary RAM address in user mode; more on this later.
- **System:** System mode has the same registers as user mode but is privileged. It is normally used only for the transition from supervisor to user mode, to set the user mode registers without switching to an unprivileged mode
  .
- **Supervisor**: This is the mode in which the processor begins execution. It is also the mode in which the kernel runs. When the processor is in a non-privileged mode, it can switch to Supervisor mode using a "Supervisor Call Instruction (SVC)."
- **IRQ:** The processor switches to IRQ mode when an interrupt is executed.

- **FIQ:** The FIQ (Fast Interrupt Request) mode corresponds to the IRQ mode, but the difference is that the FIQ mode is intended only for those interrupts that must be executed with the shortest possible latency. It can also override normal interrupts in this context.
- **Abort:** The processor enters abort mode when either a data abort or a prefetch abort exception is triggered. A Data Abort is triggered when the processor requests an address for which it lacks permissions or which does not exist. A Prefetch Abort is triggered when the processor attempts to decode an instruction but cannot read it from RAM, which can occur for the same reasons as a Data Abort.
- **Undefined:** Undefined mode is selected when the processor triggers an *Undefined Exception*. This is triggered when an instruction cannot be decoded or is invalid.

Registers

The figure below shows the various registers of the Arm Cortex. R0–R12 are so-called “general-purpose registers,” meaning they can be used for anything.
The “Arm Procedure Call Standard” (AAPCS) defines R0–R3 and R12 as “scratch registers.” This means that any function executed does not need to restore these registers to their original state. Instead, they are used to pass parameters and return values to other functions. The registers R4–R11 are referred to as “callee-saved” registers; this means that if a function uses one of these registers, it restores the register’s value to its original state.
The *AAPCS* guarantees that each function can be written independently of the others and ensures thereby that no registers are damaged .

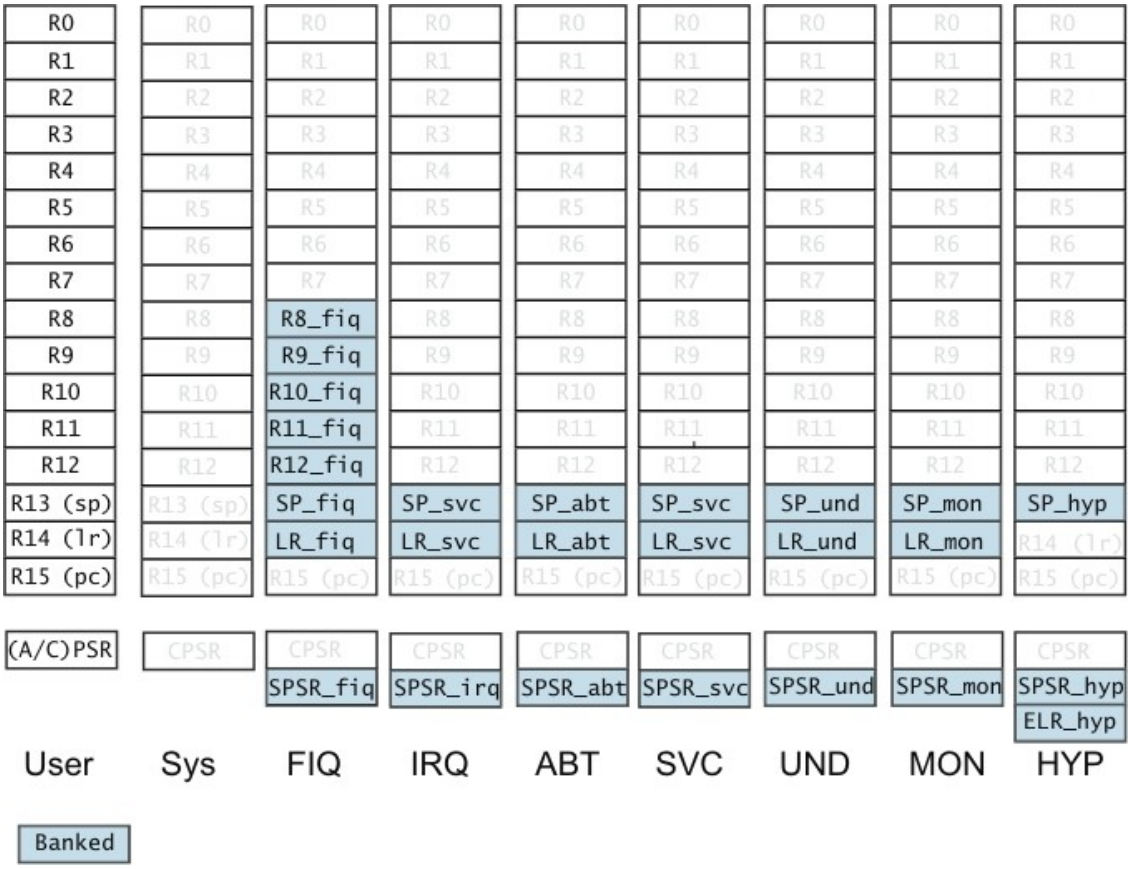


Figure 18. ARMV7a Registers
https://developer.arm.com/documentation/den0013/d/ARM-Processor-Modes-and-Registers/Registers

The **stack pointer register (sp)** is responsible for storing the position of the stack pointer. As can be seen in the figure below, in most modes it is “banked,” meaning that each mode has its own stack to prevent corruption of other modes’ stacks, particularly during exceptions.
The **Link Register (lr)** is responsible for storing the return address of a function; this address usually points to the next instruction following the function call instruction. Like the stack pointer, the link register is also banked, since in the event of an exception, the address that triggered the exception and to which the program must jump after the exception is resolved
is stored in the link register, and this should not overwrite the link register of the previous mode.
The **Program Counter Register (pc)** stores the address of the currently executing instruction.
The **Current Program Status Register (CPSR)** stores, among other things, the current processor mode, as well as flags used for branching, whether interrupts are enabled, and other details such as the processor’s current execution state—though I won’t go into further detail on that here.
When an exception occurs, the **Saved Program Status Register (SPSR)** saves the processor’s CPSR as it was before the exception occurred. This is particularly important for restoring the CPU mode and branching flags to their previous state.

Coprocessor 15

The Armv7 architecture also includes the so-called “Coprocessor 15,” which provides access to the “System Control Registers”—the registers used to control certain aspects of the CPU, including the cache, the MMU, exceptions, and other aspects of the controller

Vectors

Normal Vector offset	High vector address	Non-secure	Secure	Hypervisor[a]	Monitor
0x0	0xFFFF0000	Not used	Reset	Reset	Not used
0x4	0xFFFF0004	UNDEFINED instruction	UNDEFINED instruction	UNDEFINED instruction from Hyp mode.	Not used
0x8	0xFFFF0008	Supervisor Call	Supervisor Call	Secure Monitor Call	Secure Monitor Call
0xC	0xFFFF000C	Prefetch Abort	Prefetch Abort	Prefetch Abort from Hyp mode.	Prefetch Abort
0x10	0xFFFF0010	Data Abort	Data Abort	Data Abort from Hyp mode,	Data Abort
0x14	0xFFFF0014	Not used	Not used	Hyp mode entry	Not used
0x18	0xFFFF0018	IRQ interrupt	IRQ interrupt	IRQ interrupt	IRQ interrupt
0x1C	0xFFFF001C	FIQ interrupt	FIQ interrupt	FIQ interrupt	FIQ interrupt

Figure 19. ARMv7a Exception Vector

The vectors of the Arm Cortex A8 are located at address *0x00000000* by default, but can also be configured as so-called high vectors, in which case they are located at *0xFFFF0000*. However, this is not practical for the AM335, as it has no RAM at these addresses that it can access. However, the coprocessor can be used to remap the address of the vectors to any desired address

The MMU

The Memory Management Unit, or MMU for short, is a component of every modern chip that uses a general-purpose operating system such as Linux, macOS, or FreeBSD. The MMU's primary function is to protect specific regions of RAM from unauthorized access; for example, no user application should be able to read the memory of the kernel or other processes. In addition, the MMU is capable of "remapping" specific regions of RAM. That is, when the processor requests a specific RAM address, e.g., *0x402f0400*, the MMU translates this address and, depending on the configuration, accesses a different physical RAM address, e.g., 0x00000000. This concept is called "virtual memory" and is the cornerstone of many operating systems. Among other things, this is very useful when a processor's memory layout is not very efficient for a specific application; by "remapping" the RAM, this problem can be circumvented.
MMU translations are performed directly in hardware, which makes the operations very fast. In most cases, the translations are also stored in the L1 or L2 cache, resulting in even faster translations.
On smaller microcontrollers, such as those based on the Armv7-M architecture, only a so-called MPU (Memory Protection Unit) is used, which can set permissions for different regions but cannot implement virtual memory.

Page-tables

With an MMU, the entire memory is usually divided into so-called pages, where each "page" has its own permissions; thus, each process in the kernel can be assigned a page to which only that process has access. During translation, only the upper 12 or 20 bits are used to translate the address. The LSBs are not altered in this process.

Level 1

In the ARM Cortex, memory is typically divided into 4,096 distinct regions, each 1 MB in size.
To perform the translation, it takes the 12 MSBs and uses them as an offset for the page table to find the correct page table entry.

Among other things, the processor provides the option to use two different L1 page tables, one for higher address spaces and the other for lower ones. This is usually used to have different memory layouts for the kernel and user processes. In my kernel, however, only one L1 page table was used.

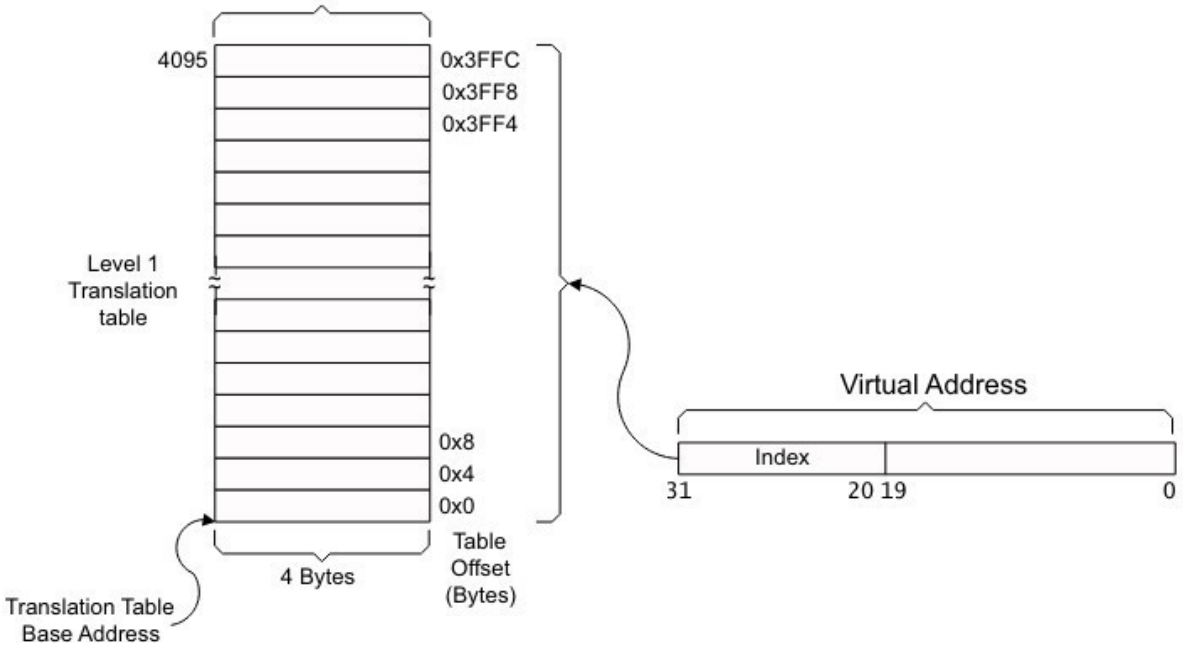


Figure 20. ARMV7a L1 Translation Table
https://developer.arm.com/documentation/den0013/d/The-Memory-Management-Unit/First-level-address-translation

A distinction is made here between four different page table entries.

- **Fault:** Triggers a Data Abort Exception
- **Pointer:** This is a pointer to another translation table; more on this later
- **Section:** Translates the upper 12 bits into another 12 bits.
- **Supersection:** Supersections do not translate 1 MB regions like normal sections, but rather 16 MB regions; however, this is used less frequently.

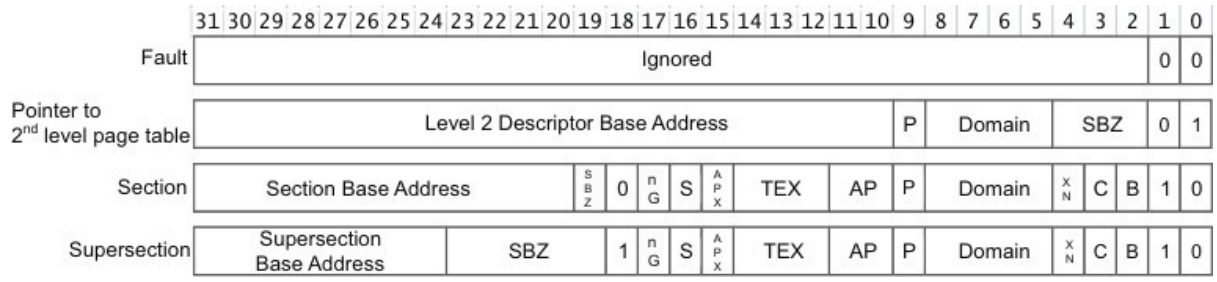


Figure 21. ARMV7a L1 Page Table Entry
https://developer.arm.com/documentation/den0013/d/The-Memory-Management-Unit/First-level-address-translation

Level 2

If a page table contains a pointer to a Level 2 page table, the pointer is stored within the page table. The Level 2 page table must be aligned to 1 KB, meaning that the 10 least significant bits (LSBs) must all must be zero and contains 256 page table entries. Among other things, the page table can be divided into sections of 4 KB or 64 KB.

As with Level 1 page table entries, a distinction is made between different types of page table entries:

- **Fault:** Triggers a Data Abort Exception
- **Large page:** Translates only the upper 16 bits, so “large pages” have a size of 64 KB. However, these must be repeated 16 times.
- **Small page:** Translates the 20 most significant bits, resulting in a page size of 4 KB.

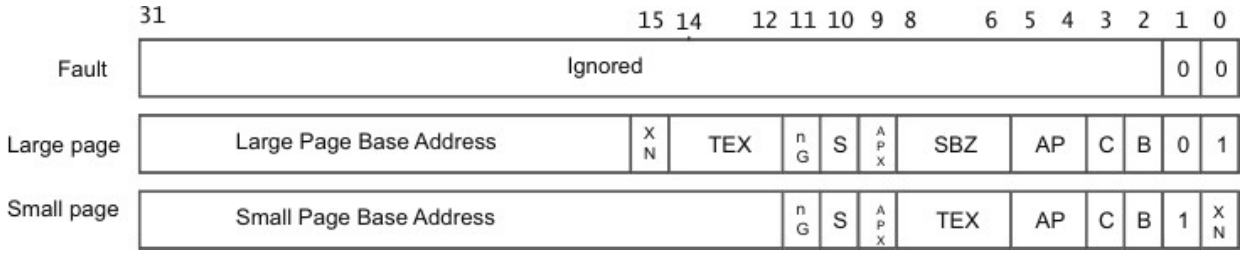


Figure 22. ARMV7a L2 Page Table Entry
https://developer.arm.com/documentation/den0013/d/The-Memory-Management-Unit/Level-2-translation-tables

In the figures above, you can see a number of other bits and sections of the various page table entries; these are primarily permission and memory ordering attributes. However, I do not wish to go into detail about them here.

Translation Lookaside Buffer

The Translation Lookaside Buffer is a cache of the most recently executed translations. Thanks to the TLB, the MMU does not have to perform a translation table walk for every memory request, but can simply retrieve the necessary values from the cache. Since most variables are typically located in the same region of RAM—which is usually cached—it can be assumed that the cache is hit for every memory request, significantly improving the speed
of RAM accesses.
The virtual and physical addresses, as well as the memory attributes, are stored there. In addition, the ASID is also stored there, which can be understood as the current process ID. This allows multiple page translations with the same virtual address to be stored, because in this case the TLB only returns the page translation that has the same ASID as the one currently stored in *Coprocessor 15*. As a result, the TLB does not have to be invalidated every time a context switch occurs.

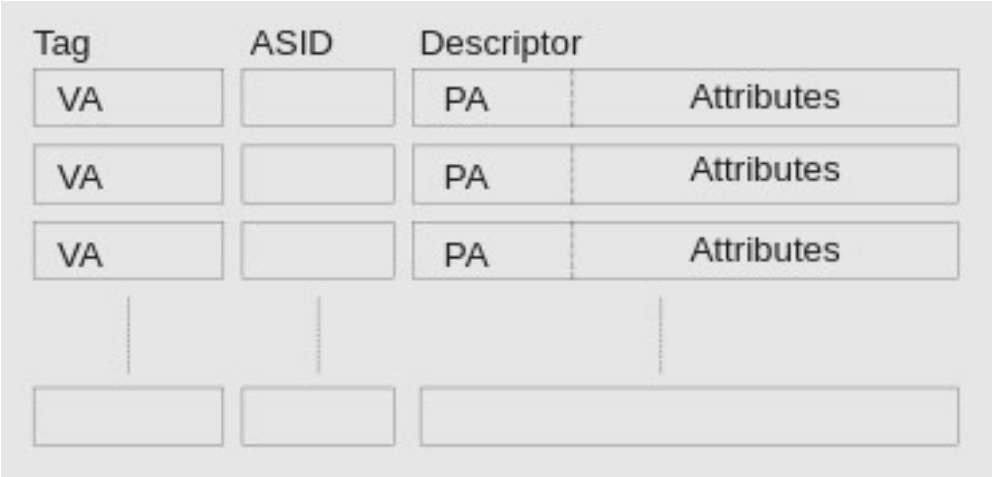


Figure 23. ARMV7a Translation Lookaside Buffer
https://developer.arm.com/documentation/den0013/d/The-Memory-Management-Unit/The-Translation-Lookaside Buffer

Booting

The AM335 is not booted via USB, as is the case with other microcontrollers such as the Arduino Nano or the ESP
, but rather with a MicroSD card, which must be written with the kernel.
The AM335 requires a “Table of Contents” and a “GP Header” before the code. The “Table of Contents” provides general information about the image, while the header specifies the image’s size and its location in memory.

The following command in Linux was used to flash the image to the SD card:

```
sudo dd if=./out/rom.img of=/dev/sda oflag=direct bs=4M status=progress
```

Compilation

If you want to compile a binary for a controller that does not run on any operating system such as Linux or Mac, you must use a linker. This is because when compiling for Linux, for example, the linker is integrated directly into the compiler, since the compiler already knows where to where to place the code. However, when programming for so-called “bare metal,” the compiler does not know where to place the code. Therefore, a linker script must be added during compilation so that the compiler knows where to place the code.
Note: We refer here to “placing code,” but this simply means which addresses the compiler assigns to the various instructions to enable branching to the correct address.

Linker Script

In the following section, I would like to explain the linker script in a bit more detail.
The *MEMORY* section describes the processor's memory layout. *RWX* indicates that this region is writable, readable, and executable, while *ORIGIN* specifies the starting address in memory. Entry refers to the symbol at which the controller starts.
The *SECTIONS* section will later describe where the various symbols are placed. Here is a brief description of the symbols:

- **Text:** This is where all the code is placed. Here, the .text._start symbol was placed before all the other symbols to ensure that the _start symbol is at the very beginning
- **BSS:** All global variables that have not yet been initialized are stored here
- **Data:** All global variables that are initialized with a value are stored here.

Linker.ld

```
MEMORY {
    sram (rwx) : ORIGIN = 0x402F0400, LENGTH = 0xFFFF
}

ENTRY(_start)
SECTIONS {
    .text : {
        *(.text._start)
        *(.text*)
    } > sram

    .bss : {
        *(.bss*)
    } > sram

    .data : {
        *(.data*)
    } > sram

    ...

    _end = .;
}
```

Cargo

Cargo is Rust's package manager. It must be configured for compilation on the AM335. Below is the Cargo configuration file for compilation.
First, the "target" is set; this describes the controller's architecture and CPU so that Cargo knows which features of the controller it can use. For example, it can incorporate floats into the code if it knows that this controller has an FPU.
Next, the linker is set; this is part of the *arm-none-eabi* toolchain, which includes additional tools such as an assembler, compiler, and others. The linker arguments are then set in the *rustflags*. These include, among other things, the location of the linker script, as well as the various "object files" into which some assembly files have been converted.
N o t e : Object files are assembly files that have already been compiled but do not yet know which region of memory they will be placed in; they still need to be incorporated into the linker

.cargo/config.toml

```
[build]
target = "armv7a-none-eabi"

[target.armv7a-none-eabi]
linker = "arm-none-eabi-ld"
rustflags = [
    "-C", "link-arg=-Tkernel/boot/linker.ld",
    "-C", "link-arg=kernel/out/setup.o",
    "-C", "link-arg=kernel/out/exceptions.o",
    "-C", "link-arg=kernel/out/interrupts.o",
    "-C", "link-arg=kernel/out/software_interrupts.o",
    "-C", "link-arg=kernel/out/kernel.o",
]
```

Build Script

The entire process is then integrated into a GNU Makefile to automate the build process. Here, the assembly files are compiled first, then the entire project is compiled with Cargo, and finally combined with the TOC and the header, which can then be flashed to the SD card.

Makefile

```
build:
  arm-none-eabi-gcc -mcpu=cortex-a8 -c src/asm/setup.S -o out/setup.o
  arm-none-eabi-gcc -mcpu=cortex-a8 -c src/asm/interrupts.S -o out/interrupts.o
  arm-none-eabi-gcc -mcpu=cortex-a8 -c src/asm/exceptions.S -o out/exceptions.o
  arm-none-eabi-gcc -mcpu=cortex-a8 -c src/asm/software_interrupts.S -o out/software_interrupts.o
  arm-none-eabi-gcc -mcpu=cortex-a8 -c src/asm/kernel.S -o out/kernel.o
  cargo build --release
  cp ../target/armv7a-none-eabi/release/kernel out/kernel.elf
  arm-none-eabi-objdump -d out/kernel.elf > out/kernel.dump
  arm-none-eabi-nm out/kernel.elf > out/kernel.map
  arm-none-eabi-objcopy out/kernel.elf -O binary out/boot.bin
  cat boot/toc.bin boot/header.bin out/boot.bin > out/rom.img

```

Startup

Before the kernel initializes the peripherals or other components, the stack, caches, and exceptions must be initialized first. Here, the kernel uses the *`setup_modes`* routine to set up the stack for the respective modes. Next, the caches are initialized, followed by the initialization of the vectors containing the handlers for the various exceptions.

src/main.rs

```
pub fn _start() {
    unsafe {
        setup_modes();
        setup_caches();
        setup_exceptions();
    }

    ...
```

Stack setup

First, the stack pointer for IRQ mode is set. Using the *MSR* instruction, a value can be written to the *CPSR* register. The suffix of *the CPSR* indicates that only the interrupt flags and the mode can be overwritten. The stack pointer is then overwritten with the address of the *`irq_stack_end`* symbol, which is defined in the linker script. The same is then done for supervisor mode. Finally, interrupts are enabled, and the
processor remains in supervisor mode. Finally, the processor jumps to the contents of the LR register. The attentive observer will notice here that the user mode stack pointer is not set. This is always performed before a task switch, making it unnecessary here.

src/asm/setup.S

```
1  setup_modes:
2      @ IRQ mode
3      mov r0, #0xD2
4      msr cpsr_c, r0
5      ldr sp, =irq_stack_end
6
7      @ Supervisor mode
8      mov r0, #0xD3
9      msr cpsr_c, r0
10     ldr sp, =stack_end
11
12     @ Enable IRQ
13     mov r0, #0x53
14     msr cpsr_c, r0
15
16     bx lr
17
```

Cache setup

To activate the various caches, the L1 cache must first be disabled. Next, the L1 caches and the data caches are invalidated to completely clear them. Finally, both branch prediction and D-side prefetching are enabled.

Armv7 chips contain so-called Branch Prediction Units (BPUs) that run in parallel with the CPU.
Since the CPU cannot know which of the two options it will take in an *if/else statement*, for example, it cannot cache instructions and data. The BPU can predict which of the two options is most likely and prefetch the instructions and data.
In D-Side prefetching, a hardware component assists the CPU when certain patterns are detected in data access. For example, when the CPU iterates through an array, this hardware component can recognize this and preload the data into the cache to prevent a RAM roundtrip. Since the cache is much closer to the CPU, this can lead to a significant performance improvement.

As can be seen, at the beginning of the function, the link register is pushed onto the stack and then popped into the PC register. This is necessary because a *bl* instruction (Branch with Link) overwrites the LR. Accordingly, it must be saved on the stack and is finally transferred to the PC register.
We will spare the reader the details of implementing these functions, as they are not particularly important.

src/asm/setup.S

```
1  setup_caches:
2      push {lr}
3
4      bl disable_l1_caches
5      bl invalidate_l1_caches
6      bl invalidate_data_caches
7      bl branch_prediction_enable
8      bl enable_d_side_prefetch
9
10     pop {pc}
```

Exceptions Setup

The setup of the vectors is very important, as they are defined by the handlers for interrupts and software interrupts, without which the kernel could not function. The vectors must be aligned to the nearest 5-bit boundary. To initialize the vectors, their address must be loaded into the *r0* register and then written to the “Vector Base Address Register” (VBAR), which is located in Coprocessor 15. Finally, a so-called “Data Synchronization Barrier” is executed, which ensures that all data accesses are complete before program execution resumes.

src/asm/exceptions.S

```
1  .align 5
2  vectors:
3      b _start
4      b undefined_handler
5      b handle_swi
6      b fetch_abort_handler
7      b data_abort_handler
8      b .
9      b irq_handler
10     b .
11
12 setup_exceptions:
13     ldr r0, =vectors
14     mcr p15, 0, r0, c12, c0, 0
15     dsb
16
17     bx lr
```

AM335 Hardware

In the following chapter, I would like to discuss the hardware of the AM335. This refers to the various implementations of the AM335-specific modules. These include modules such as timers and interrupts; however, since their implementation is specific to the AM335, they are grouped in the same chapter as typical peripherals such as GPIO or I2C.

Interrupts

Unlike many other processors, the AM335 does not implement the *ARM General Interrupt Controller (GIC)*. Instead, a dedicated interrupt controller was developed specifically for the AM335. This controller has a total of 128 different interrupts, which can be triggered by the various modules of the AM335. Among other things, each interrupt has a priority, which can have a value ranging from 0 to 127.

Activation

To enable an interrupt, the mode and priority must first be written to the *INTC_ILR*. The mask bit for the corresponding interrupt is then cleared, which enables the interrupt.

src/interrupts.rs

```rust
pub fn enable_interrupt(interrupt: Interrupt, mode: Mode, priority: u8) {
    let interrupt_number = interrupt as u32;

    let addr = INTC + INTC_ILR + (4 * interrupt_number);
    let enable_fiq = match mode {
        Mode::IRQ => 0,
        Mode::FIQ => 1,
    };
    let bank = match InterruptBank::new(interrupt_number) {
        Some(bank) => bank,
        None => return,
    };

    write_addr(addr, enable_fiq | (priority << 2) as u32);
    set_bit(INTC + bank.get_mir() + MIR_CLR_OFFSET, interrupt_number % 32);
}
```

Handler

As already described here, the interrupt handler is set via the vectors.

At the beginning of the interrupt handler, 4 is subtracted from the link register, since when the handler is called, the link register is advanced by 4 addresses beyond the next instruction. Accordingly, 4 must be subtracted from it. Subsequently, all registers of the previous task are saved to the stack. !Note: Since the link register is banked, the link register
in interrupt mode can be overwritten without any issues.
Next, the *SPSR*—which contains the *CPSR* of the previous task—is pushed onto the stack. Then, the interrupt handler in the Rust code is called, which, among other things, ensures that the correct interrupt handler is executed. After that, a so-called *Data Instruction Barrier (dsb)* is inserted to ensure that all memory operations are completed. Finally, the *SPSR* and the registers are restored. It is important to note that if a ^ is appended as a suffix to the *ldmfd* instruction and a *pc* is present in the list of registers, the SPSR is also restored in
the CPSR is copied, thereby restoring the *CPSR* of the previous task.

src/asm/interrupts.S

```
1  irq_handler:
2      sub lr, lr, #4
3      stmfd sp!, {r0-r12, lr}
4
5      mrs r11, spsr
6      push {r11}
7
8      bl handle_interrupt
9
10     mov r0, #0
11     mcr p15, #0, r0, c7, c10, #4
12
13     pop {r11}
14     msr spsr, r11
15
16     ldmfd sp!, {r0-r12, pc}^
```

One of the tasks of the Rust handler is to read the current interrupt number, execute the corresponding interrupt, and finally clear the interrupt to signal to the processor that the interrupt has been successfully completed.

src/interrupts.rs

```rust
static mut INTERRUPT_HANDLERS: &mut [fn(); 128] = &mut [noop; 128];

#[no_mangle]
fn handle_interrupt() {
    let interrupt = current();
    execute(interrupt);
    clear();
}
```

Clocks

Every peripheral in the AM335 controller requires a clock for the module's internal logic. This is called the "Interface Clock." This clock must be explicitly enabled, as clocks also consume power and are therefore disabled by default.
Here, a distinction is made, among other things, between the WKUP and the PER domains.
The WKUP domain typically contains one module per peripheral, which is powered by that peripheral. These modules are primarily used when the controller enters low-power mode, as they can wake the controller back up by sending specific signals. However, these modules were not used here because low-power mode was not employed.
The PER domain contains all other peripherals of the controller.

The various clocks can be activated by writing *0x2* to the respective "Clock Module" register to enable the clock. For this purpose, a module was written for the kernel that allows each clock to be activated via a simple API.

The kernel can activate the clock for a specific module using the following routine:

src/peripherals/gpio.rs

```
1 clock::enable(clock::FuncClock::Gpio1);
```

Pinmuxing

Since the processor supports a wide variety of peripherals, it cannot provide a single pin with just one function for each peripheral. Therefore, the AM335 implements
pin multiplexing for up to 8 different functions on a single pin. Using the *Control Module*, one of up to 8 different functions can be selected for each pin, and it is also possible to define whether the input of a pin should be enabled. Additionally, you can specify whether a particular pin should have a pull-up, pull-down resistor, or no resistor
should be present at the output.

src/pinmux.rs

```rust
pub fn set_pin_mode(offset: u32, mode: u32, input_enable: bool, pull_resistor: PullResistor) {
    let control_module = CONTROL_MODULE_BASE + offset;

    write_addr(
        control_module,
        mode | (pull_resistor.to_mask() << 3) | ((input_enable as u32) << 5),
    );
}
```

GPIO

Initialization

Two clocks are required to use the GPIO module. The first is the interface clock, which, as with other modules, is necessary to control the registers. The second is the debounce clock, which automatically handles debouncing and requires a 32 kHz clock.

The GPIO module consists of one module in the WKUP domain (GPIO0) and three modules in the PER domain (GPIO1–3). Since the module contains more than one module, the GPIO pins are divided into four different “GPIO banks.” Each of these banks contains a total of 32 different pins.

Pin Mode

As with other microcontrollers, the module’s pin mode must be set at the beginning. This is done via the *GPIO_OE* register, where a set bit corresponds to an input and an unset bit corresponds to an output. It is important to note that, as with other GPIO registers, the nth bit is responsible for the nth pin of the GPIO bank.

src/peripherals/gpio.rs

```rust
pub fn pin_mode((pin, bank): GpioPin, mode: GpioMode) {
    match mode {
        GpioMode::Input => {
            set_bit(bank as u32 + GPIO_OE, pin);
        }
        GpioMode::Output => {
            clear_bit(bank as u32 + GPIO_OE, pin);
        }
    }
}
```

Pin writing

To write to a pin whose direction has been set to output, you must set the corresponding bit in the *GPIO_DATAOUT* register.

src/peripherals/gpio.rs

```rust
pub fn write((pin, bank): GpioPin, value: bool) {
    if value {
        set_bit(bank as u32 + GPIO_DATAOUT, pin);
    } else {
        clear_bit(bank as u32 + GPIO_DATAOUT, pin);
    }
}
```

Pin reading

To read a GPIO pin, you must read the corresponding bit from the *GPIO_DATAIN* register.

src/peripherals/gpio.rs

```rust
pub fn read((pin, bank): GpioPin) -> bool {
    read_bit(bank as u32 + GPIO_DATAIN, pin)
}
```

Interrupts

Initialization

Each GPIO module provides two interrupt lines. This is done to assign different priorities to different pins. In this implementation, however, only one interrupt line was used for simplicity. The following routines activate the interrupts for GPIO Bank 1 and assign the handler to the interrupt.

src/peripherals/gpio.rs

```rust
interrupts::enable_interrupt(Interrupt::GPIOINT1A, Mode::IRQ, 1);
interrupts::register_handler(handle_interrupts, Interrupt::GPIOINT1A);
```

Registration

To register a handler for a specific GPIO pin, the user must specify,
which event triggers the interrupt. This can be either a rising edge or a falling edge, or both. The code sets the interrupt handler in the GPIO handler array and sets the bit in *GPIO_IRQSTATUS_SET0*, which enables interrupts for the pin. Finally, the respective bits for the corresponding edges are set.

src/peripherals/gpio.rs

```rust
pub fn register_interrupt(pin: u32, interrupt: GpioInterrupt, handler: fn()) {
    unsafe {
        GPIO_INTERRUPT_HANDLERS[pin as usize] = handler;
    }

    set_bit(GPIO1 + GPIO_IRQSTATUS_SET0, pin);

    match interrupt {
        GpioInterrupt::Rising => set_bit(GPIO1 + GPIO_RISINGDETECT, pin),
        GpioInterrupt::Falling => set_bit(GPIO1 + GPIO_FALLINGDETECT, pin),
        GpioInterrupt::Change => {
            set_bit(GPIO1 + GPIO_RISINGDETECT, pin);
            set_bit(GPIO1 + GPIO_FALLINGDETECT, pin)
        }
    }
}
```

Handling

The interrupt handler is responsible for selecting the correct handler when an interrupt is triggered and executing it. At the beginning, the handler reads the number of the triggered interrupt. The corresponding bit is set; the “Count Leading Zeros” (clz) instruction is used to determine which bit is set, and the corresponding handler is then executed based on this. Finally, the corresponding bit in *GPIO_IRQSTATUS_0* is set, which marks the interrupt as completed.
This register is a so-called “write-to-clear” register.

src/peripherals/gpio.rs

```rust
fn handle_interrupts() {
    let irq_raw = read_addr(GPIO1 + GPIO_IRQSTATUS_RAW_0);
    let number = irq_raw.trailing_zeros();

    unsafe {
        GPIO_INTERRUPT_HANDLERS[number as usize]();
    }

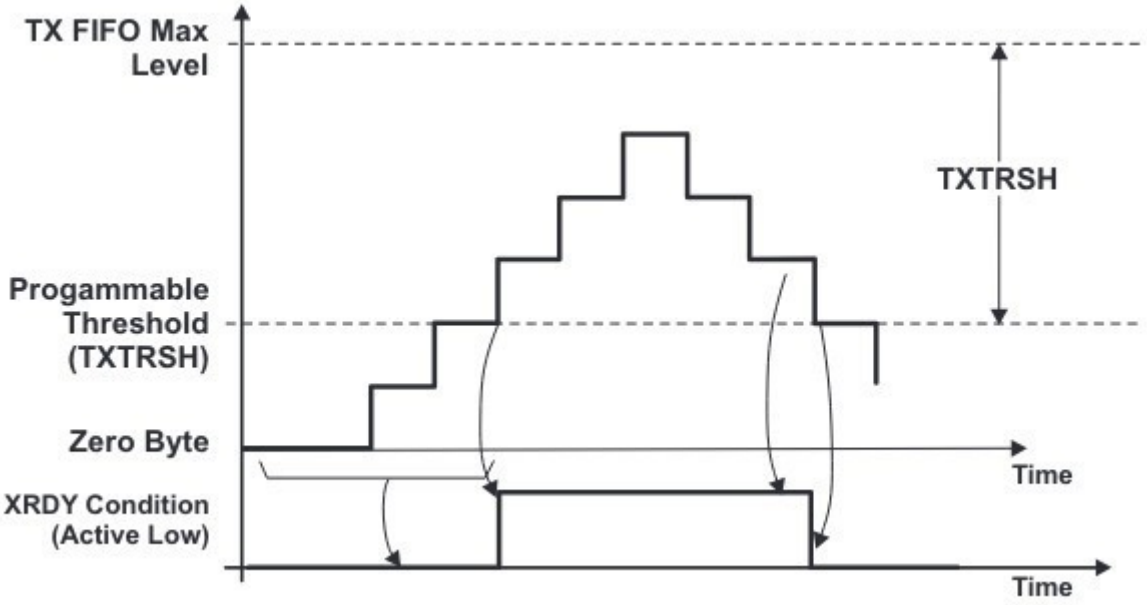
    write_addr(GPIO1 + GPIO_IRQSTATUS_0, 1 << number);
}
```

I2C

To help the reader better understand the subsequent pages regarding the I2C implementation, I would like to explain how the I2C module works in this section, as understanding it can be very difficult without this explanation. For the sake of simplicity, I will limit myself here to the transmit functionality, but the same principles apply equally to the receive functionality. It is assumed here that the reader is reasonably familiar with the I2C protocol.

To avoid the software having to send each bit of the message individually and manually, a 32-bit FIFO is implemented on the AM335 for both Rx and Tx. This has the advantage that it can be filled in advance by the software and output without further intervention. When the FIFO reaches a certain threshold, an interrupt can signal the controller, which then refills or empties it.
In such FIFOs, the threshold is usually set at the midpoint to avoid triggering interrupts too frequently, but also to ensure that the FIFO is empty before it is refilled because the software is too slow.
The following diagram illustrates the entire process. At the beginning, an interrupt is triggered with the goal of filling the FIFO. These are then sent one by one, and accordingly, the FIFO contains fewer and fewer values. When the number of values becomes smaller than *TXTRSH*, i.e., the threshold, a new interrupt is triggered to continue filling the FIFO. This process continues until all data has been sent.

Figure 24. AM335 I2C TXTRSH TRM Page 4595



Initialization

The AM335 contains a total of 3 I2C modules. One of them is located in the WKUP domain, the other two in the PER domain. The I2C2 module was used for the implementation of the I2C module. The clock for this module can be enabled in a similar manner to the GPIO module.

src/peripherals/i2c.rs

```
clock::enable(FuncClock::I2C2);
```

Next, the interrupts for the I2C module are enabled, as this module operates using interrupts.

src/peripherals/i2c.rs

```
interrupts::enable_interrupt(Interrupt::I2C2INT, Mode::IRQ, 2);
interrupts::register_handler(irq_handler, Interrupt::I2C2INT);
```

To initialize the I2C module, you must first put the module into software reset mode. Next, the I2C module's clock registers are initialized to generate a clock frequency of 100 kHz in master mode. Then, the module's own address is set in case it is addressed as a slave. Now the module can be activated and then waits until the reset is complete. Finally, the threshold to 16 .

src/peripherals/i2c.rs

```
self.soft_reset();
self.init_clocks();
self.set_own_address();
self.enable();
self.wait_reset();

self.setup_threshold();
```

Operation

The implementation for performing a specific I2C operation was implemented here using the *hal_embedded* crate (library) for the I2C module. This is used by many different libraries. By implementing the *I2C* trait, these can thus be used with my kernel. The implementation of *Receive* has not yet been implemented, but this should not pose a major problem; however, I had other priorities initially.

Transmit

First, the mode is selected; in this case, that would be the master transmitter. Next, the slave address is set, and finally, the internal buffer and the FIFO are cleared. Finally, the system waits until the bus is free, because if another device is already using the bus , , , , it should not attempt

src/peripherals/i2c.rs

```rust
for i in 0..buffer.len() {
    self.transmit_buffer.push(buffer[i]);
}

self.set_count(buffer.len() as u32);
self.ready = false;

self.start();

self.enable_interrupts(I2cMode::Transmitter);
self.wait_ready();
self.disable_interrupts(I2cMode::Transmitter);

self.stop();
```

Next, the data from the *`buffer`* parameter is copied into the internal transmit buffer. Additionally, the *`I2C_CNT`* register is set to the length of the buffer to inform the processor how many bytes are to be sent. This register is decremented with each transmission, and once it reaches zero, the operation is declared complete. The *`self.ready`* variable is then set to `false`; more on this later. Later, the start bit is set, and the necessary interrupts for transmitter mode are activated. The processor then waits until the internal *`self.ready`* variable is set to `true` by the interrupts and deactivates the interrupts for transmitter mode again. Finally, the module sends the stop bit, after which program execution continues.

Interrupts

In this section, I would like to focus solely on explaining how the transmit interrupts work. It should also be noted that this is only a minimal implementation of the I2C module, and it can be improved, particularly in terms of error handling.

src/peripherals/i2c.rs

```rust
impl I2cMode {
    fn interrupts(&self) -> &[I2cInterrupt] {
        match self {
            I2cMode::Transmitter => &[
                I2cInterrupt::XRDY,
                I2cInterrupt::XDR,
                I2cInterrupt::ARDY,
                I2cInterrupt::NACK,
            ],
            I2cMode::Receiver => &[
                I2cInterrupt::RRDY,
                I2cInterrupt::RDR,
                I2cInterrupt::ARDY,
                I2cInterrupt::NACK,
            ],
        }
    }
}
```

The following code determines which interrupts are activated for which operations. As mentioned earlier, I will focus only on those in transmitter mode here; however, those in receiver mode are conceptually similar to those in transmitter mode.

XRDY

The *XRDY* interrupt is triggered when the data to be transmitted is less than *TXTRSH* and is a multiple of *TXTRSH*.

src/peripherals/i2c.rs

```rust
if value & I2cInterrupt::XRDY as u32 != 0 {
    for _ in 0..TRANSMIT_THRESHOLD {
        self.write_data();
    }

    write_addr(self.base() + I2C_IRQSTATUS, I2cInterrupt::XRDY as u32);
    return;
}

```

XDR

The XDR interrupt is part of the so-called “draining feature” of the I2C core. This is triggered when the length of the remaining data to be sent is less than *TXTRSH*. This has the advantage that the software does not have to check how much data it can still send with every *XRDY* interrupt. This is thus minimized to just one interrupt.

src/peripherals/i2c.rs

```rust
if value & I2cInterrupt::XDR as u32 != 0 {
    for _ in 0..self.transmit_bytes_available() {
        self.write_data();
    }

    write_addr(self.base() + I2C_IRQSTATUS, I2cInterrupt::XDR as u32);
    return;
}
```

ARDY

The *ARDY* interrupt is triggered when all data has been sent and the operation is considered complete. The interrupt sets the internal *self.ready* variable to *true*, causing the processor to exit the *wait_ready* function and re-enable interrupts for transmitter mode, as described here.

src/peripherals/i2c.rs

```
1 if value & I2cInterrupt::ARDY as u32 != 0 {
2     self.ready = true;
3
4     write_addr(self.base() + I2C_IRQSTATUS, I2cInterrupt::ARDY as u32);
5     return;
6 }
```

NACK

If the acknowledgment bit is not sent by the slave, a *NACK* interrupt is triggered. This sets the *self.error* variable.

src/peripherals/i2c.rs

```rust
if value & I2cInterrupt::NACK as u32 != 0 {
    self.error = Some(I2cError::Nack);
    self.ready = true;

    write_addr(self.base() + I2C_IRQSTATUS, I2cInterrupt::NACK as u32);
}
```

After the transmit operation is completed as described here, this section checks whether the *self.error* variable is *true*. If it is true, the operation is stopped and the error is returned.

src/peripherals/i2c.rs

```rust
if let Some(error) = self.error {
    self.stop();
    self.disable();

    self.error = None;

    return Err(error);
}
```

Timer

The AM335 contains a total of 7 *DMTimer* modules and one *DMTimer_1ms* module. The latter module allows for greater accuracy, but it was not implemented because it was not a high priority.

The following implementation uses the so-called *Auto Reload* mode. In this mode, the timer starts with an initial value and increments the counter at a defined interval. As soon as the counter reaches the maximum (*0xFFFF_FFFF*), it is reset to the value of *the load register*, which normally corresponds to the initial value. An interrupt is triggered every time the processor needs to reset the counter value.

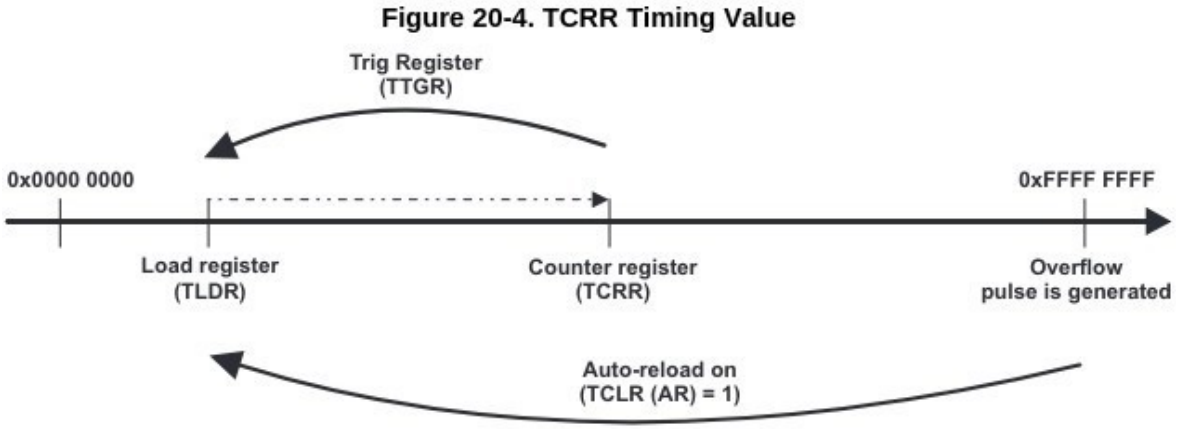


Figure 25. AM335 Timer TCRR Register TRM Page 4442

By correctly defining the start value/load register, the programmer can trigger an interrupt at any desired interval. The clock can be adjusted here. In the following implementation, only the 32 kHz clock was used; however, any clock could be used, provided it is applied to the TCLKIN pin via the hardware. For simplicity, a prescaler value of *1* was used here. In the table below, it is important to note that the 32 kHz clock does not correspond exactly to an interval
rather than 31.25 µs, it produces an interval of approximately 30.5 µs. However, this isn't necessarily important for my project, which is why I used the standard *DMTimer* was used here instead of the more precise *DMTimer_1ms*.

Clock	Prescaler	Resolution	Interrupt Period Range
32.768 KHz	1 (min)	31.25 us	31.25 us to ~36h 35m
	256 (max)	8 ms	8 ms to ~391d 22h 48m
25 MHz	1 (min)	40 ns	40 ns to ~171.8s
	256 (max)	10.24 us	~20.5 us to ~24h 32m

Figure 26. AM335 Timer Interval Range

Initialization

As with any peripheral, the clock must be enabled at the start.

src/internals/timer.rs

```
self.timer.clock().enable()
```

Next, the *Timer Counter Register* and the *Timer Load Register* are loaded with the reload value. This value can be calculated using the following formula, among other methods.
$(0xFFFFFFFF - TLDR + 1) \times timer\ Clock\ period \times Clock\ Divider\ (PS)$. If you now want an interval of, say, 1 ms, you must set the *TLDR* register to the value *0xFFFFFFE0*.

src/internals/timer.rs

```
write_addr(self.timer.address() + TIMER_LOAD, self.reload);
write_addr(self.timer.address() + TIMER_COUNTER, self.reload);
```

Next, the interrupts for auto-reload mode are enabled, followed by registering the interrupt handler.

src/internals/timer.rs

```
self.irq_enable();

interrupts::register_handler(Self::handle_timer_irq, self.timer.interrupt());
interrupts::enable_interrupt(self.timer.interrupt(), interrupts::Mode::IRQ, 0);
```

Interrupts

At the beginning of the interrupt handler, the active interrupt is read. The active *DMTimer* is then selected based on the interrupt number. From there, the interrupt flag is reset and the timer's handler is executed.

src/internals/timer.rs

```rust
fn handle_timer_irq() {
    let interrupt = interrupts::current();

    if let Some(interrupt) = interrupt {
        let timer = get_timer(interrupt);

        if let Some(timer) = timer {
            timer.irq_acknowledge();
            (timer.handler)();
        }
    }
}
```

Modularity

During the implementation of the timer module, emphasis was also placed on the system's modularity, making it possible to use six different timers, each with its own interval.

src/internals/timer.rs

```rust
pub enum DmTimer {
    Timer2,
    Timer3,
    Timer4,
    Timer5,
    Timer6,
    Timer7,
}
```

This was achieved by storing a global array containing all the different timers. Using the *`register_timer`* function, a timer can be added to the array, noting that it is initialized in the process. Using the *`get_timer`* function, the currently running *DMTimer* can be retrieved via the interrupt.

src/internals/timer.rs

```rust
static mut TIMERS: &mut [Option<Timer>; 6] = &mut [const { None }; 6];

pub fn register_timer(dm_timer: DmTimer, reload: u32, handler: fn()) {
    let timer = Timer::new(dm_timer, reload, handler);
    unsafe { TIMERS[dm_timer as usize] = Some(timer) }
}

pub fn get_timer(interrupt: Interrupt) -> &'static Option<Timer> {
    let dm_timer = match DmTimer::try_new(interrupt) {
        Some(dm_timer) => dm_timer,
        None => return &None,
    };

    unsafe { &TIMERS[dm_timer as usize] }
}
```

Kernel

In the chapter on the kernel, I would like to describe the structure of the kernel and explain kernel-specific features, such as multitasking and paging. This includes, among other things, the structure of the codebase and the addition of external tasks, which are then executed simultaneously. Among other things, the following description of the kernel also covers the implementation of user and kernel space, which prevents user code from directly accessing kernel functions.

Sysclock

The following code is the implementation of the Sysclock module. This module is responsible for keeping track of the system time. It initializes Timer2 with a value of *0xFFFF FFE0*, which guarantees an interval of 1 ms. In the interrupt handler, the Sysclock is incremented by 1. Subsequently, the *yield_task* routine is executed every 10 ms; more on this later.

The *`millis`* routine can then be used to read the value of the system clock.

src/internals/sysclock.rs

```rust
pub fn initialize() {
    timer::register_timer(DmTimer::Timer2, 0xFFFF_FFE0, interrupt_handler);
}

static mut SYS_CLOCK: u32 = 0;

fn interrupt_handler() {
    unsafe { SYS_CLOCK += 1 };

    if unsafe { SYS_CLOCK } % 10 == 0 {
        unsafe { yield_task() };
    }
}

pub fn millis() -> u32 {
    unsafe { SYS_CLOCK }
}
```

Syscalls

Syscalls, software interrupts, or *Supervisor Calls (SVC)* in the ARM architecture are used to perform privileged actions in an unprivileged mode. Since the processor cannot switch modes on its own in an unprivileged mode, syscalls are required.

Various parameters can be passed to the syscall, which it uses to select the appropriate action and then execute the desired action or return a specific value.

For example, the following instructions can be used to execute syscall number 0 with the first parameter as the value
5. The syscall number is used here to select the type of syscall, while the other registers are used as parameters.

```
1 mov r0, #5
2 svc #0×0
```

Handler

The handler for software interrupts is structured similarly to that of hardware interrupts. However, there are some important differences. First, unlike with hardware interrupts, the link register points directly to the next instruction. Additionally, the syscall handler takes registers *r0–r3* as parameters and can also return a value in the *r0* register.

To understand the first part of the handler, you need to know that when a *supervisor* issues *a call*, as shown in the figure above, an immediate must be appended, which can be up to
24 bits long and will be encoded in the instruction.
The handler takes the value of the previous instruction—that is, the one that triggered the SVC—and clears the 8 MSBs, leaving only the syscall number.

Next, the corresponding registers are saved to the stack, which is then stored in the *r1* register. Afterward, 8 bytes of space are allocated in the stack pointer to store the return values.

```
handle_swi:
    stmfd sp!, {r0-r12, lr}

    sub lr, lr, #4
    ldr r12, [lr]
    bic r12, r12, #0×FF000000

    push {r0-r3, r12}
    mov r1, sp

    sub sp, #8
    mov r0, sp

    bl swi_handler

    ldr r0, [sp, #4]
    ldrb r1, [sp]
    add sp, #8

    add sp, sp, #20

    cmp r1, #0×1
    beq exit

    str r0, [sp]

    ldmfd sp!, {r0-r12, pc}^
```

Then the Rust software interrupt handler is executed, whose task is to perform the corresponding actions and, if necessary, return a return value.

After the Rust software interrupt handler has executed, the two return values are stored in *r0* and *r1*, and the stack pointer is incremented by 8 to remove them from the stack. The stack pointer is then incremented by 20 to free up space on the
stack. If the *r1* register is 1 (i.e., true), the *exit* function is executed. Finally, the *r0* register stored on the stack is overwritten with the return value of the syscall.

The *exit* function was added here to make it possible to remain in supervisor mode after an *SVC*, i.e., to stop the execution of the current task. In doing so, the mode is switched to supervisor, and the stack pointer is incremented by 56, since registers *r0–r12* and the *PC* were not popped from the stack. Finally, the processor jumps to the `*kernel_loop*` function, which then selects and executes the next task; more on that later.

```
1 exit:
2     msr cpsr_c, #0x53
3     add sp, sp, #56
4
5     b kernel_loop
```

Rust Handler

Signature

The following signature is used for the Rust syscall handler. As discussed earlier, the stack pointer for the return value is stored in *r0*, and the arguments are passed via a stack pointer in *r1*.

src/kernel.rs

```rust
extern "C" fn swi_handler(frame: &TrapFrame) -> SyscallReturn {
```

The figures below illustrate the data structures mentioned. The *TrapFrame* struct stores the various registers in its fields. The *SyscallReturn* struct, on the other hand, stores whether the handler should exit user mode, as well as one of the various return values.

src/kernel.rs

```rust
#[repr(C)]
struct TrapFrame {
    r0: u32,
    r1: u32,
    r2: u32,
    r3: u32,
    r12: u32,
}

#[repr(C)]
struct SyscallReturn {
    exit: bool,
    value: SyscallReturnValue,
}

#[repr(C)]
pub union SyscallReturnValue {
    pub millis: u32,
    pub gpio_read: bool,
    pub i2c_write: I2cError,
    pub alloc: *mut u8,
    pub none: (),
}
```

Logic

In *the swi_handler*, the *TrapFrame* is then converted into a Syscall Enum. Subsequently, the corresponding logic of the syscall is executed, and a *SyscallReturn* object is returned by each of these syscall handlers.

src/kernel.rs

```
1  extern "C" fn swi_handler(frame: &TrapFrame) -> SyscallReturn {
2      let syscall: Syscall = match frame.try_into() {
3          Ok(syscall) => syscall,
4          Err(_) => panic!("invalid syscall"),
5      };
6
7      match syscall {
8          Syscall::Exit => {
9              ...
10
11             SyscallReturn::exit()
12         ...
```

Syscall Enum

The Syscall Enum is used in the kernel to store the various parameters of the syscalls. Another advantage is that by using this enum, the user no longer has to manually write assembly code to execute the syscall; this can be done automatically by the syscall.

src/kernel.rs

```rust
pub enum Syscall<'a> {
    Exit,
    Yield {
        sp: u32,
        pc: u32,
        until: Option<u32>,
    },
    Millis,
    GpioRead {
        pin: GpioPin,
    },
    GpioWrite {
        pin: GpioPin,
        value: bool,
    },
    I2cWrite {
        address: u8,
        data: &'a [u8],
    },
    Panic,
    Alloc {
        layout: Layout,
    },
    Dealloc {
        ptr: *mut u8,
        layout: Layout,
    },
}

```

The following function executes the system call, while also storing the various return values and returning them as well.

src/kernel.rs

```rust
impl Syscall<'_> {
    pub fn call(self) -> Option<SyscallReturnValue> {
        match self {
            Syscall::Exit => unsafe {
                asm!("svc 0x0", options(noreturn));
            },
            Syscall::Yield { sp, pc, until } => unsafe {
                asm!("svc 0x1", in("r0") sp, in("r1") pc, in("r2") until.unwrap_or(0), options(noreturn));
            },

        ...
```

Multitasking

Since the AM335 has only one core, if the user wants to execute multiple tasks simultaneously, they must allocate a certain amount of time to each task before executing another task. This process, also known as “CPU scheduling,” was implemented in my operating system.
When implementing CPU scheduling, each process was assigned an execution time of 10 ms. This is made possible by assigning each task its own stack, on which the *CPSR* and the registers of that task are subsequently saved.

The kernel implements both *cooperative* and *preemptive multitasking.* In *preemptive multitasking*, the CPU switches tasks at regular intervals; this is typically achieved using timers and interrupts. In *cooperative multitasking*, however, the active task voluntarily yields execution; this can occur, for example, when the task is in *a sleep state* or when it is currently waiting for an interrupt to complete.

Preemptive multitasking

In the sysclock interrupt handler, the *`yield_task`* routine is called every 10 ms, which sets the *`yielded`* flag. If this flag is set at the end of the interrupt handler, the registers and the *CPSR* are not reset as usual, but are saved on the stack of the previous task, after which the next task is executed.

src/kernel.rs

```rust
fn interrupt_handler() {
    unsafe { SYS_CLOCK += 1 };

    if unsafe { SYS_CLOCK } % 10 == 0 {
        unsafe { yield_task() };
    }
}
```

The following section is located at the end of the interrupt handler code. In the upper section, a check is performed to determine whether the previous mode was Supervisor, meaning the kernel was not currently executing a task. In this case, the registers are restored as usual and the mode is switched.
If this is not the case, the *`yielded`* flag is checked; if it is set, the context of the previous task is saved; otherwise they the tabs are restored as usual.

src/asm/interrupts.S

```
    pop {r11}
    msr spsr, r11
    and r11, r11, #0b11111
    cmp r11, #0b10011
    beq return_interrupt

    ldr r0, yielded
    cmp r0, #0x1
    beq store_context

return_interrupt:
    ldmfd sp!, {r0-r12, pc}^
```

The *store_context* routine is responsible for saving the task's registers to the task's stack so that they can be restored. At the beginning, the registers are popped from the stack here; it is important to note that the suffix ^ was not appended, which is why the

The mode is not switched. It is also important to note that some registers, such as the link register and the *SPSR*, are stored in temporary variables because they are banked and therefore cannot be used by other modes. The mode is then switched to System Mode, which shares the registers with User Mode, whereupon the registers of the previous task are saved on the task stack, as is the *SPSR*—more precisely, the *CPSR*—of the previous task.
Finally, a software interrupt is initiated, which is tasked with passing the task's stack pointer and the next instruction at the address stored in the Link Register as parameters to the syscall.

src/asm/interrupts.S

```
store_context:
    ldmfd sp!, {r0-r12, lr}

    str lr, next_pc

    push {r0}
    mrs r0, spsr
    str r0, temp_spsr
    pop {r0}

    msr cpsr_c, #0×DF
    stmfd sp!, {r0-r12, lr}

    mov r0, #0×0
    str r0, yielded

    ldr r0, temp_spsr
    push {r0}

    mov r0, sp
    ldr r1, next_pc
    mov r2, #0×0
    svc #0×1

```

In the syscall handler, the current task is then selected, and the stack pointer and return address are saved there. Additionally, the task's status is set to *"Stored,"* which signals to the scheduler that it must also restore the registers. Finally, an *exit* is returned, causing the current code to stop executing and jump back to the *kernel_loop*.

src/kernel.rs

```rust
Syscall::Yield {
    sp,
    pc,
    until: None,
} => {
    let scheduler = scheduler();
    if let Some(task) = scheduler.current() {
        task.context.pc = pc;
        task.context.sp = sp;
        task.state = TaskState::Stored;
    }

    scheduler.cycle();

    SyscallReturn::exit()
}

```

Cooperative multitasking

In *cooperative multitasking*, the processor voluntarily yields execution until a specific action has been performed. This process is also called *yielding*. In this project, *yielding* has so far only been used for the *sleep* routine, but could be further expanded, such as
For example, until an I2C operation is complete.

To *yield* the current task, you must call the following function, which takes a single argument representing the time for which the task's execution should be deferred. First, the relevant registers are saved to the stack pointer, along with the *CPSR*, and then added as parameters for the syscall via the various registers.

src/sysclock.rs

```
yield_task:
    stmfd sp!, {{r0-r12, lr}}

    mov r2, r0

    mrs r0, cpsr
    push {{r0}}

    mov r0, sp
    mov r1, lr
    mov r2, r2
    svc #0x1
```

src/sysclock.rs

```
fn yield_task(ms: u32);
```

The following code is the handler for the yield system call when it is initiated by the process itself. As in *preemptive multitasking*, it saves the stack pointer and the address of the next instruction, sets the status to "Waiting," and adds the time until which the task's execution is to be postponed.

src/kernel.rs

```rust
Syscall::Yield {
    sp,
    pc,
    until: Some(until),
} => {
    let scheduler = scheduler();
    if let Some(task) = scheduler.current() {
        task.context.pc = pc;
        task.context.sp = sp;
        task.state = TaskState::Waiting { until };
    }

    scheduler.cycle();

    SyscallReturn::exit()
}
```

Task- -Restore

Once the next process has been selected, and if it has already been executed previously, its *CPSR* and registers must be restored to their original state. To do this, the *CPSR* is first popped from the stack and then written to the *SPSR*, which is subsequently used as the *CPSR*. It is important to note that the CPSR is popped from the stack via register *R0*
task. Next, the addresses at which the task's execution is to resume are stored on the stack to be written to the *PC* register. Then the stack pointer is set to system mode, and the registers are popped from the stack. The mode is then switched back to Supervisor Mode, and the *PC* stored on the Supervisor Mode stack is popped, after which the system switches to User Mode, since this is stored in the *SPSR*.

src/asm/interrupts.S

```
restore_context:
    ldmfd r0!, {r2}
    msr spsr, r2

    push {r1}

    msr cpsr_c, #0×DF
    mov sp, r0
    pop {r0-r12, lr}
    msr cpsr_c, #0×D3

    ldmfd sp!, {pc}^
```

src/tasks.rs

```
fn restore_context(sp: u32, pc: u32) → !;
```

Scheduling

As mentioned earlier, the scheduling technique used here is known as round-robin scheduling. In this approach, none of the processes has a priority, as is common in many operating systems; instead, after one process is executed, the next process is executed, and so on, until the scheduler reaches the last process, at which point it starts over from the beginning. This method is not optimal for most systems that need to distinguish between different priorities; however, it was implemented because it is one of the simplest scheduling methods.

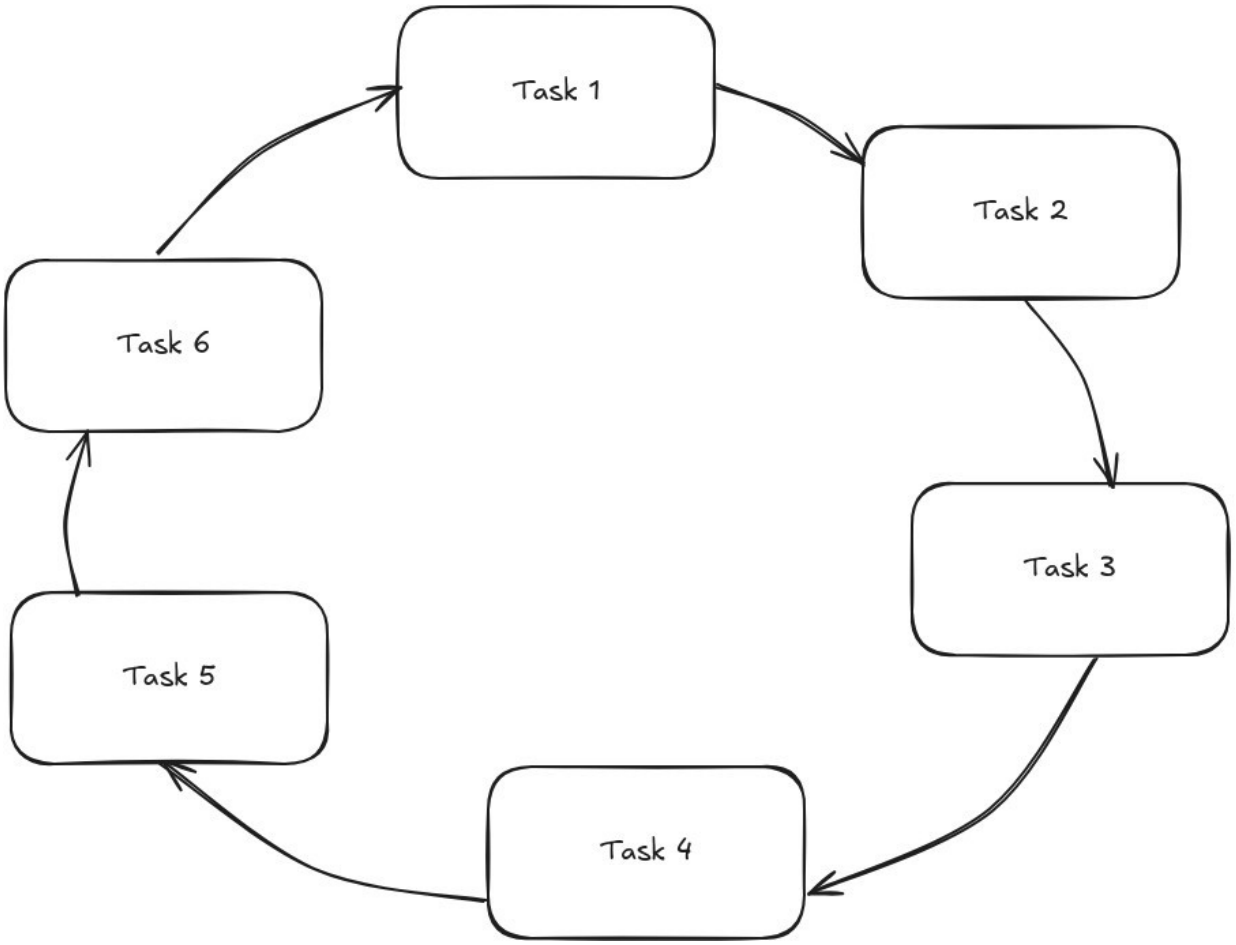


Figure 27. Round-Robin Scheduling

The following code is responsible for executing the next executable task. At the beginning, it receives the next task to be executed. This task is then set as the current task and executed. If the task's status is *"Ready,"* it means the task has not yet been executed, and therefore no registers are saved on its stack. However, if the status is *"Stored,"* the task has already been executed, meaning its registers are saved on the stack and must be popped again.

src/internals/tasks.rs

```rust
pub fn switch(&mut self) {
    let next_task_id = match self.next_task() {
        Some(task) => task.id,
        None => return,
    };

    self.current_index = Some(next_task_id);

    let task = self.task_mut(next_task_id);

    match task.state {
        TaskState::Ready => {
            task.state = TaskState::Running;
            unsafe {
                switch_context(task.context.sp, task.context.pc);
            }
        }
        TaskState::Stored => {
            task.state = TaskState::Running;
            unsafe {
                restore_context(task.context.sp, task.context.pc);
            }
        }
        _ => {}
    }
}

```

The *switch_context* function thus saves the stack pointer in system mode at the beginning and then switches back to supervisor mode, whereupon the *SPSR* is set to user mode, and finally the *r1* register is moved to the *pc* register, whereby the *s* suffix of the *mov* instruction has the same effect as the ^ suffix in the *ldmfd* instruction, namely that the *SPSR* is copied into the *CPSR*.

src/asm/kernel.S

```
switch_context:
    msr cpsr_c, #0×DF
    mov sp, r0
    msr cpsr_c, #0×D3

    mov r2, #0×50
    msr spsr_c, r2

    movs pc, r1
```

The `*next_task*` function is responsible for finding the next executable task. To do this, it iterates through all tasks; if a task is executable, it returns it. When the index reaches the original index again, the search is completed and `*None*` is returned.

src/internals/tasks.rs

```
fn next_task(&mut self) → Option<&mut Task> {
    let initial_index = self.current_index.unwrap_or(0);
    let mut index = initial_index;

    loop {
        let current_task = self.task_mut(index);
        if current_task.executable() {
            return Some(current_task);
        }

        index = (index + 1) % MAX_TASKS;
        if index == initial_index {
            break;
        }
    }

    None
}
```

The `*kernel_loop*` function is responsible for repeatedly switching processes and is called, among other things, when a process terminates execution via either *cooperative* or *preemptive multitasking*.

src/kernel.rs

```
pub fn kernel_loop() {
    loop {
        let scheduler = scheduler();
        scheduler.switch();
    }
}
```

Paging

Kernel paging ensures that no user process can access the kernel's memory. This is made possible by the fact that only privileged modes can access the kernel's memory region and that of the peripherals. Since the various tasks run in user mode, which is non-privileged, they cannot access these regions. With paging, each process can be compiled independently without having to make assumptions about its environment. This is possible because, although the code for each task is stored in different locations, when a task is executed, this block of memory is mapped to *0x0*, as is its stack.
In the following implementation, the L1 page table was used to set access permissions for peripheral and kernel memory. The L2 page table, on the other hand, which was implemented only once, was used solely for the various memory regions of the individual processes.

Initialization

First, the *L1* and *L2 translation tables* are initialized. Next, the *TTBCR* and *TTBR0* are initialized, with the *TTBCR* defining the relationship between the *TTBR0* and *TTBR1*. The *TTBR0*, on the other hand, stores the address of the L1 page table. Then the TLB is invalidated and the domains are initialized; however, I will not go into detail about these, as they were not used and have also been deprecated since *ARMv7*. Finally, the *MMU* itself is enabled, at which point the address translations take effect.

src/internals/mmu/setup.rs

```rust
pub fn initialize() {
    unsafe {
        l1::initialize();
        l2::initialize();
        initialize_ttbcr();
        initialize_ttbr0();
        invalidate_tlb();
        setup_domains();
        enable_mmu();
    }
}
```

L1 page table

The L1 page table is stored globally in memory. A *struct* must be used for this purpose to align the page table to 16 KB. By default, all sections of RAM are filled with false page table entries.

src/internals/mmu/l1.rs

```rust
pub static mut LEVEL1_PAGE_TABLE: L1PageTable = L1PageTable::new();

#[repr(align(16384))]
pub struct L1PageTable(pub [u32; PAGE_TABLE_SIZE]);

impl L1PageTable {
    const fn new() -> Self {
        L1PageTable([FAULT_PAGE_TABLE_ENTRY; PAGE_TABLE_SIZE])
    }
}
```

During initialization, the regions containing the kernel and peripherals are then filled with section page table entries, which have privileged access permissions so that no user task can access these regions.

src/internals/mmu/l1.rs

```rust
const PERIPHERAL_MEMORY: Range<u32> = 0x4400_0000..0x8000_0000;
const KERNEL_MEMORY: Range<u32> = 0x4020_0000..0x4040_0000;

pub fn initialize() {
    enable_memory_range(KERNEL_MEMORY, AccessPermissions::Privileged);
    enable_memory_range(PERIPHERAL_MEMORY, AccessPermissions::Privileged);
}

fn enable_memory_range(range: Range<u32>, permissions: AccessPermissions) {
    for page in range.step_by(PAGE_SIZE as usize) {
        let section = L1SectionPageTableEntry::new(page, permissions);
        unsafe {
            LEVEL1_PAGE_TABLE.0[page as usize >> PAGE_SIZE_BITS] = section.into();
        }
    }
}
```

L2 page table

Like the L1 Page Table, the L2 Page Table must also be stored globally; however, unlike the L1 Page Table, it only needs to be aligned to 1 KB.

src/internals/mmu/l2.rs

```rust
#[no_mangle]
static mut LEVEL2_PAGE_TABLE: L2PageTable = L2PageTable::new();

#[repr(align(1024))]
pub struct L2PageTable([u32; PAGE_TABLE_SIZE]);

impl L2PageTable {
    const fn new() -> Self {
        L2PageTable([L2_FAULT_PAGE_TABLE_ENTRY; PAGE_TABLE_SIZE])
    }
}
```

To initialize it, the first element of the L1 Page Table is replaced with a pointer to the L2 Page Table, so that all address requests from *0x0 to 0x100000* now map to the corresponding L2 Page Table entries.

src/internals/mmu/l2.rs

```rust
pub fn initialize() {
    let l1_pointer = L1PointerTableEntry::new(&raw mut LEVEL2_PAGE_TABLE);

    unsafe {
        LEVEL1_PAGE_TABLE.0[0] = l1_pointer.into();
    }
}
```

Creating an L2 page table

To create an L2 page table entry, the programmer must add the virtual address and the ASID. Since the virtual address is always 0 in the following kernel implementation, an additional global array is used to track the pages that have already been used, ensuring they cannot be used by another process.

src/internals/mmu/l2.rs

```rust
pub struct L2SmallPageTableEntry {
    asid: Option<u32>,
    virtual_address: u32,
    physical_address: u32,
    permissions: AccessPermissions,
}

impl L2SmallPageTableEntry {
    pub fn try_new(virtual_address: u32, asid: Option<u32>) -> Option<Self> {
        let current_index =
            (0..PAGE_TABLE_SIZE as u32).find(|&i| unsafe { !USED_PAGES[i as usize] })?;
        unsafe {
            USED_PAGES[current_index as usize] = true;
        }
        let offset = current_index << PAGE_SIZE_BITS;

        Some(L2SmallPageTableEntry {
            asid,
            virtual_address: virtual_address & !0xFFF,
            physical_address: BASE_ADDRESS + offset,
            permissions: AccessPermissions::Full,
        })
    }

    ...
```

For an L2 page table entry to be usable, it must be “registered.” During this process, the ASID of the page table entry overwrites that of the processor and places the L2 page table entry in the L2 page table corresponding to the virtual address. Finally, a *data synchronous barrier* and an *instruction synchronous barrier* are added to ensure that all operations are completed.
When the page table entry is no longer in use, a fault entry is set at its virtual address, and the corresponding region is marked as unused. Finally, the entry for this page table entry in the TLB is invalidated, while the remaining entries in the TLB remain unchanged. Finally, an *isb* and a *dsb* are also added here.

src/internals/mmu/l2.rs

```rust
impl L2SmallPageTableEtnry {
    ...

    pub fn register(&self) {
        self.set_asid();

        unsafe {
            LEVEL2_PAGE_TABLE.0[self.virtual_address as usize >> PAGE_SIZE_BITS] = self.into();

            asm!("dsb", "isb");
        }
    }

    pub fn unregister(&self) {
        unsafe {
            LEVEL2_PAGE_TABLE.0[self.virtual_address as usize >> PAGE_SIZE_BITS] =
                L2_FAULT_PAGE_TABLE_ENTRY;
            USED_PAGES[(self.physical_address - BASE_ADDRESS) as usize >> PAGE_SIZE_BITS] = false;
        }

        self.invalidate_tlb();

        unsafe {
            asm!("dsb", "isb");
        }
    }

    fn invalidate_tlb(&self) {
        unsafe {
            asm!("mcr p15, 0, {mva}, c8, c7, 1", mva = in(reg) (self.virtual_address & !0×FFF) |
self.asid.unwrap_or(0));
        };
    }

    ...
```

Application

Paging is used when allocating regions of RAM. Each executable task is stored in a region of RAM, while the stack and heap of that task are stored in a separate region.

When a new task is created, a code page table entry and a data L2 page table entry are created, after which the code is copied to the code page. Next, both page table entries are added to the task, the status is set, and the stack pointer and program counter are initialized. Finally, the heap is initialized, but more on that later.

src/internals/tasksrs

```rust
pub fn create_task(&mut self, code: &[u8]) -> Option<usize> {
    let task_id = self.task_with_state(TaskState::Terminated)?.id;

    let code_page = L2SmallPageTableEntry::try_new(CODE_PAGE_LOCATION, Some(task_id as u32))?;
    let data_page = L2SmallPageTableEntry::try_new(DATA_PAGE_LOCATION, Some(task_id as u32))?;

    let dest = code_page.start() as *mut u8;
    unsafe {
        ptr::copy_nonoverlapping(code.as_ptr(), dest, code.len());
    }

    let task = self.task_mut(task_id);
    task.code_page = code_page;
    task.data_page = data_page;
    task.state = TaskState::Ready;
    task.context.sp = task.data_page.end();
    task.context.pc = task.code_page.start();
    task.allocator
        .init(task.data_page.start() as usize, task.data_page.end() as usize - STACK_GUARD);
    Some(task.id)
}
```

Adding Tasks

To add various tasks, a function was simply added to the scheduler and executed at the start of kernel development. However, after the kernel had integrated features such as paging, I decided to handle this outside the kernel code. Since, unfortunately, my operating system does not allow me to add various programs at runtime as many other operating systems do, I was limited here to adding tasks at compile time.
Each program is compiled individually, and the resulting binary is then placed in the *`programs`* folder. A *procedural macro* reads this folder and inserts the binaries into the code. *Procedural macros* allow code to be executed before the main code is compiled. This code can access Rust's standard library, enabling it to read the contents of the *`programs`* folder and embed the compiled programs directly into the main program.

First, the *`programs`* folder is read and then converted into an array containing the respective contents of the compiled programs. These are then embedded into the program as an array using the *`quote`* crate.

include_programs/src/lib.rs

```rust
#[proc_macro]
pub fn include_programs(_input: TokenStream) -> TokenStream {
    let current_dir = env::current_dir().unwrap();
    let program_dir = current_dir.join(PathBuf::from("kernel/programs"));

    let program_files = fs::read_dir(program_dir)
        .expect("No programs directory is present")
        .flatten()
        .filter_map(|file| {
            let path = file.path();

            if path.is_file() {
                let contents = fs::read(path).unwrap();
                return Some(contents);
            }

            None
        })
        .collect::<Vec<_>>();

    let tokens: Vec<_> = program_files
        .iter()
        .map(|inner_vec| {
            quote! {
            &[#(#inner_vec), *]
            }
        })
        .collect();

    let expanded = quote! {
        &[
            #(
                #tokens
            ),*
        ]
    };

    expanded.into()
}
```

The array returned by the macro is stored globally. A separate task is then created for each of these programs.

src/main.rs

```
1  static PROGRAMS: &[&[u8]] = include_programs!();
2
3  #[no_mangle]
4  pub fn _start() {
5
6      ...
7
8      for program in PROGRAMS {
9          create_task(program);
10     }
11
12     ...
```

User binary

The user binary uses a slightly modified linker script similar to that of the kernel. In this case, the binary is located at *0x0* and has a maximum length of 4 KB. Depending on the program, this may be a limitation; however, due to a lack of priority, no solution has been implemented to address this issue.

user/boot/linker.ld

```
MEMORY {
    ram (rwx) : ORIGIN = 0x0, LENGTH = 0x1000
}

ENTRY(_start)
SECTIONS {
    .text : {
        *(.text._start)
        *(.text*)
    } > ram

    .bss : {
        *(.bss*)
    } > ram

    .data : {
        *(.data*)
    } > ram

    _end = .;
}
```

Structure

To use various functions in the user code, these must be provided by a crate or library. A naming convention for libraries is followed here, and the crate is therefore named “libfenix.” However, since there is code used by both the kernel and the “libfenix” crate, a “shared” crate is also implemented, which primarily provides structs for communication between the two.
A Cargo workspace is used to contain these modules in a single repository. This allows multiple crates to be grouped together, and one of the main advantages is that the dependencies will use the same version; however, this is not relevant to the following implementation.

The various settings for the workspace can be configured in the root *Cargo.toml file*, and the different “members” of the workspace can be configured here, with the “resolver” specifying how various features of crates are compiled—though this has no impact here. Among other things, the following workspace configuration also includes the “include_programs” crate, which contains the macro for embedding the programs, and the “user” crate, which is responsible for building the various programs.

Cargo.toml

```toml
[workspace]
members = [
    "libfenix",
    "include_programs",
    "user",
    "kernel",
    "shared",
]
resolver = "2"
```

Heap

Only a minimal heap was used in this kernel. For simplicity, a so-called buffer allocator was used here, which, unlike a normal allocator, cannot free up memory. The buffer allocator has an internal pointer that shifts the pointer forward with each allocation, but consequently cannot reuse memory that has already been used.

shared/src/allloc/heap.rs

```rust
pub struct BumpAllocator {
    heap_start: usize,
    heap_end: usize,
    next: CriticalSection<usize>,
}

unsafe impl GlobalAlloc for BumpAllocator {
    unsafe fn alloc(&self, layout: core::alloc::Layout) -> *mut u8 {
        let mut current = self.next.lock();
        let alloc_start = align_up(*current, layout.align());
        let alloc_end = alloc_start.saturating_add(layout.size());

        if alloc_end > self.heap_end {
            ptr::null_mut()
        } else {
            *current = alloc_end;
            alloc_start as *mut u8
        }
    }

    unsafe fn dealloc(&self, _ptr: *mut u8, _layout: core::alloc::Layout) {}
}
```

Kernel

This buffer allocator is stored in `global` and uses the *`global_allocator`* attribute. This attribute allows objects from the standard library, such as *`Vec`*, *which* require a heap, to be used. However, this applies only to the kernel code.

src/heap.rs

```
pub fn initialize() {
    let allocator = &raw mut ALLOCATOR;

    unsafe {
        (*allocator).init(
            &heap_start as *const usize as usize,
            &heap_end as *const usize as usize,
        );
    }
}

#[global_allocator]
static mut ALLOCATOR: BumpAllocator = BumpAllocator::new();

extern "C" {
    static heap_start: usize;
    static heap_end: usize;
}

```

The *`heap_start`* and *`heap_end`* variables are set by the linker script, with the heap having a size of 4 KB.

boot/linker.ld

```
SECTIONS {
    ...

    . = ALIGN(4);
    heap_start = .;
    . += 4K;
    heap_end = .;

    ...
}
```

Libfenix

The allocator is also defined in the *libfenix* library. However, allocation and deallocation are left to the kernel. Each task has its own heap, which also uses a BufferAllocator.

libfenix/src/alloc.rs

```rust
#[global_allocator]
static ALLOCATOR: Allocator = Allocator {};

struct Allocator {}

unsafe impl GlobalAlloc for Allocator {
    unsafe fn alloc(&self, layout: core::alloc::Layout) -> *mut u8 {
        let syscall = Syscall::Alloc { layout };
        unsafe { syscall.call().unwrap().alloc }
    }

    unsafe fn dealloc(&self, ptr: *mut u8, layout: core::alloc::Layout) {
        let syscall = Syscall::Dealloc { ptr, layout };
        syscall.call();
    }
}

```

This heap is initialized when a task is created, with its start corresponding to the start of the data page and its end at the end of the data page, minus a stack guard, which ensures that the stack data is not overwritten.

src/tasks.rs

```rust
pub fn create_task(&mut self, code: &[u8]) -> Option<usize> {

    ...

    task.allocator
        .init(task.data_page.start() as usize, task.data_page.end() as usize - STACK_GUARD);

    ...

}
```

The system call handles the allocation and deallocation of the various tasks. It takes the allocator of the current task and uses it to perform the allocation, after which the pointer returned by the allocator is returned by the system call.

src/kernel.rs

```rust
Syscall::Alloc { layout } => {
    let scheduler = scheduler();
    if let Some(task) = scheduler.current() {
        let ptr = unsafe { task.allocator.alloc(layout) };
        return SyscallReturn::value(SyscallReturnValue { alloc: ptr });
    }

    SyscallReturn::none()
}
Syscall::Dealloc { ptr, layout } => {
    let scheduler = scheduler();
    if let Some(task) = scheduler.current() {
        unsafe { task.allocator.dealloc(ptr, layout) };
    }

    SyscallReturn::none()
}
```

Project Management

The project management section includes all the documents required for our project. This included, among other things, the project proposal, which was written at the outset and then submitted to the relevant professors, who subsequently approved the project. The requirements specification outlines all the project functions that were intended to be operational upon project completion. The Gantt chart, on the other hand, aids in time management and indicates whether the project is on schedule.

Projektantrag

Seite 1/2

Ansuchen für ein Maturaprojekt, Fassung September 2024

Projekttitel:	MicroOS	
Projektleiter:	Felix Salcher	
Klasse und Schuljahr:	5. BEL	Schuljahr 2024 / 2025

Logo des Projektes:



Kurze Beschreibung des Projektes:

Das Ziel dieses Projektes ist es, ein minimalistisches Betriebssystem bzw. ein Kernel zu schreiben, welches die Grundfunktionen wie Paging, Multitasking, CPU Exceptions implementiert. Hierbei wird die Platine selbst erstellt, welche unter anderem eine CPU beinhaltet, aber auch ein externes RAM-Modul, da der Prozessor auch mehr Prozesse ausführen sollte. Das Kernel wird in der Programmiersprache Rust geschrieben. Zudem solllten auf der Platine auch einige LEDs und Taster sein, um den Controller besser testen zu können.

ARM CORTEX A53

Benotungsrelevante Meilensteine des Projektes:

- Meilenstein 1: Platinenabgabe in PW… (1. Semester)
- Meilenstein 2: Booten einer minimalen Version (2. Semester)
- Meilenstein 3: Externe Kommunikation über GPIO Pins (2. Semester)
- Meilenstein 4: Paging, Multitasking, CPU Exceptions (2. Semester)
- Meilenstein 5: Abgabe des Projektes in PW… (2. Semester)

Figure 28. Project Proposal Page 1

PAG 1:Martin De Tomaso

Der Projektleiter

PAG 2: Ivan Huber

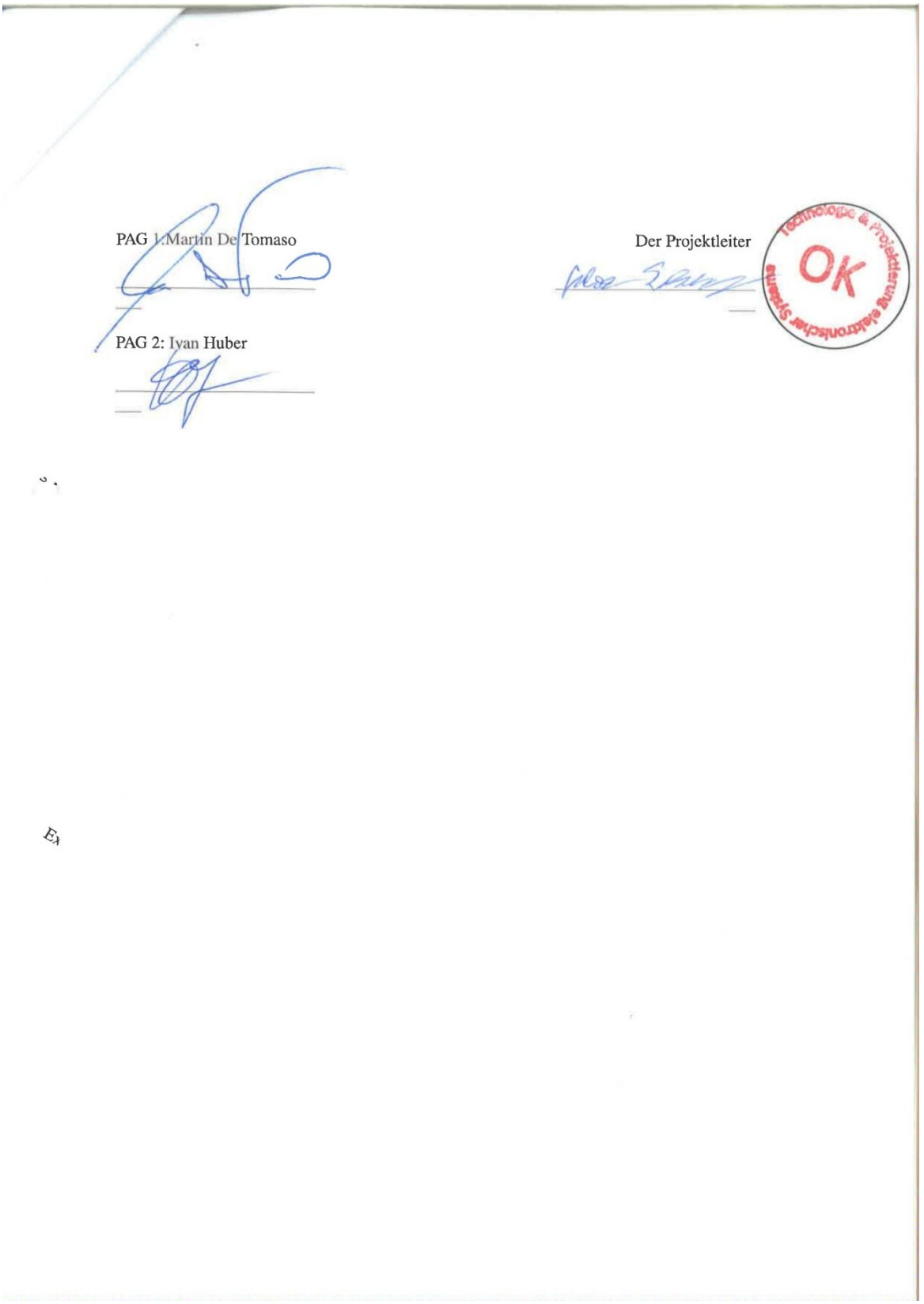


Figure 29. Project Proposal Page 2

Pflichtenheft

Projektbeschreibung:

Das Ziel dieses Projektes ist es, ein minimalistisches Betriebssystem bzw. ein Kernel zu schreiben, welches die Grundfunktionen wie Paging, Multitasking, CPU Exceptions implementiert. Hierbei wird die Platine selbst erstellt, welche unter anderem eine CPU beinhaltet, aber auch ein externes RAM-Modul, da der Prozessor auch mehr Prozesse ausführen sollte. Das Kernel wird in der Programmiersprache Rust geschrieben. Zudem sollten auf der Platine auch einige LEDs und Taster sein, um den Controller besser testen zu können

Platine:

Auf der Platine sollte der Arm Cortex-A53 Controller vorhanden sein. Dieser verwendet ein externes RAM Modul, damit er mehr Speicher zur Verfügung hat, nicht nur einige Kb, welche im Controller selbst integriert sind.

Auf der Platine sollten zudem noch einige Pinheads vorhanden sein, um externe Sensoren oder andere Geräte anzuschließen, welche anschließend durch den Controller gesteuert werden können. Außerdem werden noch einige Taster und LEDs auf der Platine integriert, um die Funktionalität der Platine direkt testen zu können, ohne externe Geräte anzuschließen. Hierbei werden jeweils 3 Taster und LEDs verwendet. Zudem sollte die Platine noch eine LED enthalten, welche vom Controller nach dem Boot automatisch eingeschaltet wird, um zu zeigen, dass der Controller richtig gebootet wurde. Das Booten sollte hierbei über USB vonstatten gehen, genauer gesagt über USB-C.

Software:

Die Software wird ein Kernel beinhalten, welches die Aufgabe hat, alle Tasks zu managen, das Speicher-layout festzulegen sowie CPU Exceptions zu handlen. Die CPU Exceptions sollten so gehandelt werden, dass das Betriebssystem sich immer davon erholen kann und niemals dadurch abstürzen sollte, hierbei sollte nur das jeweilige Programm, welches die Exception hervorgerufen hatte crashen. Zudem sollte das Betriebssystem Paging implementieren, d.h. dass jeder Prozess nur auf einen bestimmten Teil im Speicher zugreifen kann und somit nicht den Speicher anderer Programme zu korruptieren. Abschließend sollte das Betriebssystem noch in der Lage sein, mehrere Tasks gleichzeitig laufen zu lassen, entweder durch physische Threads oder durch das Freischalten einer bestimmten Zeit für jeden Prozess, in dem dieser ausgeführt werden kann. Außerdem sollte man über die Software noch die verschiedenen GPIO Pins steuern können, um mit der Außenwelt auch kommunizieren zu können.

Figure 30. Specifications, Page 1

PAG 1:Martin De Tomaso

Der Projektleiter

PAG 2: Ivan Huber



Figure 31. Requirements Specification, Page 2

Gantt-diagramm

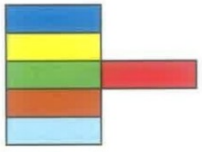
The Gantt chart was used to divide the project into various work packages. Each of these work packages is then assigned a specific timeframe within which it should be completed. During the course of the project, you then note for each work package whether it was completed within the specified timeframe. This is usually indicated by colors such as red and green.
This provides a very good overview of whether the project is on schedule.

The Gantt chart is shown on the following pages.
You can see that I fell behind schedule, particularly during the hardware development phase. This is because I didn’t decide to switch from the AM62 to the AM335 until November. This meant I practically had to start from scratch, which caused me to fall significantly behind schedule.
It is also clear that I was significantly behind in implementing paging. This is because I initially made the mistake of assuming that it would be easier to start with paging first. The reason for this is that paging usually only makes sense in the context of multitasking. That is why I began by implementing multitasking and later added the paging functionality.

Figure 32. Gantt Chart Page 1

Projekt "MicroOS" - 5BEL – Salcher Felix



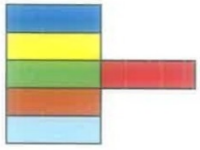


Obere Zeile: In Blau wird die Zeit eingetragen, die für den jeweiligen Working Package vorgesehen ist.
Pufferzeit (Reserve)
Untere Zeile: Vorzu händisch nachtragen. Grün: Ok, im Zeitplan Rot: Über dem Zeitplan
Meilenstein
Ferien



Projekt "MicroOS" - 5BEL – Salcher Felix

	Jahr 2024			Jahr 2025								
	PW13 9.12-15.12	PW14 16.12-22.12	PW15 23.12-29.12	PW16 30.12-05.01	PW17 06.01-12.01	PW18 13.0-19.01	PW19 20.01-26.01	**PW20** **27.01-2.02**	PW21 03.02-09.02	PW22 10.02-16.02	PW23 17.02-23.02	PW24 24.02-02.03
WP8: Erstellung minimalen Kernels												
WP9: Booten minimales Kernel auf MC												
WP10: Ansprechung GPIO Pins												
WP11: CPU Exceptions												
W12: Paging						Minimales Kernel		GPIO Pins				



Obere Zeile: In Blau wird die Zeit eingetragen, die für den jeweiligen Working Package vorgesehen ist.
Pufferzeit (Reserve)
Untere Zeile: Vorzu händisch nachtragen. Grün: Ok, im Zeitplan Rot: Über dem Zeitplan
Meilenstein
Ferien

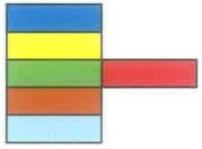


Figure 33. Gantt chart, page 2

Figure 34. Gantt Chart Page 3

Projekt "MicroOS" - 5BEL – Salcher Felix

	Jahr 2025											
	PW25 03.03-09.03	PW26 10.03-16.03	PW27 17.03-23.03	PW28 24.03-30.03	PW29 31.03-06.04	PW30 07.04-13.04	PW31 14.04-20.04	PW32 21.04-27.05	PW33 28.04-04.05	PW34 05.05-11.05	PW35 12.05-18.05	PW37 19.05-25.05
WP13: Multitasking						A						
						b						
WP14: Projektdokumentation						g				D		
						a				o		
		CPU Exceptions Paging Multitasking				b				k		
						e				u		
										m		
						P				e		
						r				n		
						o				t		
						j				a		
						e				t		
						k				i		
						t				o		
										n		



Obere Zeile: In Blau wird die Zeit eingetragen, die für den jeweiligen Working Package vorgesehen ist.
Pufferzeit (Reserve)
Untere Zeile: Vorzu händisch nachtragen. Grün: Ok, im Zeitplan Rot: Über dem Zeitplan
Meilenstein
Ferien



Cost Calculation

This chapter lists and calculates the various costs of the project to provide an overview of how much the project would have cost a company, including all expenses.

Consulting Hours

If assistance from external parties is required, these consultants must also be included in the cost calculation. A consultant’s hourly rate is calculated at €40 per hour.

Only my professor, Huber Ivan, helped me design the circuit board; no one else assisted with this project.

Date	Number of hours	Advisor's Name
11/29/2024	0.5	Huber Ivan
12/10/2024	0.25	Ivan Huber
January 9, 2025	0.75	Ivan Huber

Total consultant hours: 1.5 Cost
of external consultants: €60

Working hours

I worked on the project for 5 hours each week in the lab. Especially during the second semester, I continued working on the project very intensively at home. In some weeks, I worked up to 20–30 hours at home. In total, I worked 500 hours.
At an hourly wage of €40, the development of the project would have cost my employer €10,000.

Circuit board

The PCB was ordered from JLCPCB and assembled there as well. The PCB and its assembly cost a total of €140, including shipping. For cost reasons, it was decided to have only two PCBs assembled, as this costs over €50 per PCB.

However, it is important to note that the components used on the circuit board are already included in this price, which totals €17.08. This amount must then be subtracted from the BOM cost.

	Product	File Name	Order Number	QTY	Unit Price	Ext.Price
1	6-layer Bare Rigid Printed circuit board	maturaprojekt_Y61	Y61	3	EUR €6.76	EUR €20.29
2	Rigid Populated printed circuit board	maturaprojekt_Y61	SMT025011561535-Y61	2	EUR €55.69	EUR €111.39

Merchandise Total:	EUR €131.68
Shipping:	EUR €12.57
Merchandise Discount:	-EUR €29.07
Subtotal:	EUR €115.18
Import Taxes(22%):	EUR €25.34
Grand Total:	EUR €140.52

Figure 35. JLCPC PCB cost breakdown

BOM

Designator	Package	Quantity	Value		Unit Price($)	Price($)
R25,R26	R_0805	4	4k7		0.013	0.13
R21,R18,R16,R23,R11,R15,R22,R17,R24, ...	R_0805	17	100k		0.0009	0.05
U4	IC_TPS65217CRSLR	1	TPS65217CRSLR		2.23	4.46
L3,L2,L1	L_1008	3	2.2µH		0.167	0.167
D1	SMASeries_LTF	1	SMAJ5.0A		0.3	0.9
U1	NFBGA-324_L15.0-W15.0-R18-C18-P0.80-BL	1	AM3352BZCZ80		5.47	10.94
C77,C76	C_0805	2	18pF		0.048	0.48
C95,C4,C36,C31,C33,C34,C7,C10,C6,C32, ...	C_0805	18	10µF		0.003	0.17
C14,C15	C_0805	2	2.2µF		0.058	0.058
R27,R30,R29,R28	R_0805	4		0	0.01	0.1
C3,C86,C87,C72,C52,C94,C91,C85,C51,C49, ...	C_0805	20	100nF		0.29	0.58
U3	CRYSTAL-SMD_4P-L3.2-W2.5-BL	1	CS325S24000000ABJT		0.95	0.95
C56,C61,C58,C59	C_0805	4	1µF		0.039	0.59
R36	R_0805	1	10k, 1%		0.2	0.2
U8	TF-SMD_TF-PUSH	1	TF PUSH		0.05	0.1
D3	LED_0805	1	LED		0.17	0.85
SW1,SW2	SW_1825910-6-4	2	1825910-6		0.132	1.32
U2	UX60SC-MB-5ST	1	UX60SC-MB-5ST(80)		0.595	1.19
C50,C78,C66,C63,C69,C65,C60,C42,C64, ...	C_0402	32	100nF		0.0012	0.08
R43,R47,R49,R44,R45,R50	R_0805	6		33	0.009	0.18
R40,R39,R46,R41,R37,R20,R38,R42	R_0805	8	10k		0.009	0.18
U6	SC-70-5_L2.1-W1.3-P0.65-LS2.1-BR	1	74LVC1G07		0.1	0.5
FB1	L_0805	1	150R800mA		0.19	0.95
R14	R_0805	1	1M		0.013	0.13
R31	R_0805	1		470	0.013	0.13
C1	C_0805	1	4.7µF		0.048	0.48
J1	PinHeader_2x09_P2.54mm_Vertical	1	Conn_01x18_Pin		0.23	1.15
					Gesamt($)	27.015
					Gesamt(€)	24.06

Figure 36. BOM

Total expenses

The BeagleBone Black must also be added to the total expenses, as it was used for developing the operating system since my board did not work. The price of the board is €53.77.

BeagleBone Black	€53.77
Components	€7
Circuit board	€140.52
Labor hours	€10,000
Total	€10,201.29

As can be seen, most of the expenses are related to labor hours; the cost of hardware is negligible in this context.

List of Figures

The following section lists all images and their sources. If an image was created by the author, the source is not listed.

- Figure 1. Block diagram of the circuit board Page 15
- Figure 2. AM335 processor https://www.ti.com/product/AM3358 Page 16
- Figure 3. Processor schematic Page 18
- Figure 4. PSU schematic Page 20
- Figure 5. PSU AM335 schematic Page 21
- Figure 6. SD Card Schematic Page 23
- Figure 7. Boot Configuration Used (TRM Page 5035) Page 24
- Figure 8. Boot Pins Schematic Page 25
- Figure 9. Entire Board Page 27
- Figure 10. Top Layer of the Board Page 28
- Figure 11. PCB GND Layer Page 29
- Figure 12. PCB Layer 3 Page 30
- Figure 13. PCB Layer 4 Page 31
- Figure 14. PCB POWER Layer Page 32
- Figure 15. PCB Bottom Layer Page 33
- Figure 16. QR Code for GitHub Repository Page 36
- Figure 17. ARMV7a CPU Modes https://developer.arm.com/documentation/den0013/d/ARM-Processor-Modes-and-Register Page 40
- Figure 18. ARMV7a Register *https://developer.arm.com/documentation/den0013/d/ARM-Processor-Modes-and-Registers/Registers* Page 42
- Figure 19. ARMV7a Exception Vector https://developer.arm.com/documentation/den0013/d/Exception-Handling/Exception-priorities Page 44
- Figure 20. ARMV7a L1 Translation Table https://developer.arm.com/documentation/den0013/d/The-Memory-Management-Unit/First-level-address-translation Page 45
- Figure 21. ARMV7a L1 Page Table Entry https://developer.arm.com/documentation/den0013/d/The-Memory-Management-Unit/First-level-address-translation Page 45
- Figure 22. ARMV7a L2 Page Table Entry https://developer.arm.com/documentation/den0013/d/The-Memory-Management-Unit/Level-2-translation-tables Page 46
- Figure 23. ARMV7a Translation Lookaside Buffer https://developer.arm.com/documentation/den0013/d/The-Memory-Management-Unit/The-Translation-Lookaside-Buffer Page 65
- Figure 24. AM335 I2C TXTRSH TRM Page 4595 Page 72
- Figure 25. AM335 Timer TCRR Register TRM Page 4442 Page 72
- Figure 26. AM335 Timer Interval Range TRM Page 4437 Page 92
- Figure 27. Round Robin Scheduling Page 115
- Figure 28. Project Proposal Page 1 Page 116
- Figure 29. Project Proposal, Page 2

- Figure 30. Specifications Page 1 Page 117
- Figure 31. Requirements Specification Page 2 Page 118
- Figure 32. Gantt chart, page 1 Page 120
- Figure 33. Gantt Chart Page 2 Page 121
- Figure 34. Gantt chart, page 3 Page 122
- Figure 35. JLCPC Board Schematic Page 125

Technical Reference Manual AM335: https://www.ti.com/lit/ug/spruh73q/spruh73q.pdf

Conclusion

I am very satisfied with the development of the operating system. I was able to implement most of the features and also add some unplanned ones, such as I2C or the ability to run multiple programs.

Looking back, however, I can say that I would have done many things differently. For example, the way I communicated and structured the project. I wish I had done more in-depth research on the processor at the beginning and hadn't waited until two months into the project to choose a different one.

Another problem was that, when programming the kernel, I thought through and implemented each part individually, rather than thoroughly examining the kernel's structure from the outset. For example, synchronization methods between different processes would have been very important to address, as every operating system requires them. In addition, I should have researched the ARM architecture more thoroughly from the start, which was particularly problematic when programming the timer—I spent two weeks trying to get it to work. Ultimately, the issue was that the stack pointer for IRQ mode hadn't been set. Fortunately, with a bit of research, I was able to identify and resolve this problem.

I am very glad that I chose this project for my senior thesis. The project sparked my interest in operating systems, and I was able to learn a great deal about how they work. Among other things, I gained a complete understanding of the heap and stack, recognized the connections between compiling and linking a program, and further developed my skills in Rust.